NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY (NUST)

SCHOOL OF MECHANICAL AND MANUFACTURING ENGINEERING (SMME)

DEPARTMENT OF AEROSPACE ENGINEERING (AE – 01)

COURSES

CONTROL SYSTEMS (AE-353)

AEROSPACE INSTRUMENTATION (AE-352)

SEMESTER: *5TH*

END-SEMESTER PROJECT REPORT

"2-WHEELED SELF-BALANCING OBSTACLE AVOIDING ROBOT"

MEMBER NAME	CMS ID
SYED SHAH UMER	463322
AHMED MUKHTAR BOKHARI	461298
ATEEQ-UR-REHMAN	465502

TABLE OF CONTENTS

ABSTRACT 5

INTRODUCTION 5

1.1 The Inverted Pendulum as an Aerospace Analog 5

1.2 Problem Definition and Classification 6

1.3 Objectives 6

2. THEORETICAL FRAMEWORK AND MATHEMATICAL MODELLING 7

2.1 Coordinate Systems and Kinematics 7

2.2 Lagrangian Dynamics Derivation 8

2.2.1 Kinetic Energy Analysis ($\mathcal{T}$) 8

2.2.2 Potential Energy Analysis (V) 9

2.3 Equations of Motion and Linearization 9

2.4 Transfer Function and State-Space 10

3. PROJECT WORKFLOW AND METHODOLOGY 11

3.1 Phase 1: Project Proposal and Mathematical Planning 11

3.2 Phase 2: Component Acquisition and Empirical Prototyping 12

3.3 Phase 3: System Integration and Development 12

3.4 Phase 4: Control Synthesis, Tuning, and Interfacing 13

3.5 Phase 5: Performance Analysis and Refinement 13

4. AEROSPACE INSTRUMENTATION AND AVIONICS ARCHITECTURE 14

4.1 Sensor Physics and Spectral Characteristics 14

4.1.1 MEMS Accelerometer Principles 14

4.1.2 MEMS Gyroscope Principles 14

4.2 Sensor Fusion: The Complementary Filter 15

4.3 Environmental Sensing: Ultrasonic Range Finding 15

4.4 Driving System: L298N Dual H-Bridge Motor Driver 16

4.5 Processing Unit: Arduino Mega2560 17

4.6 Actuation System: TT Geared DC Motor and Wheel Assembly 18

4.6.1 Electromechanical Actuator: The TT Gear Motor 18

4.6.2 Traction Interface: The 65mm Wheel 19

4.7 System Design 19

4.7.1 Circuit and Code Breakdown 20

4.8 System Parameters 27
5. CONTROL SYSTEM SYNTHESIS 27
5.1 The Inner Loop: Attitude Stabilization 27
5.2 The Outer Loop: Velocity and Position Control 28
5.3 Tuning Methodologies and Stability Analysis 29
6. SYSTEM IMPLEMENTATION: HARDWARE AND SOFTWARE FUSION 30
6.1 Hardware Architecture 30
6.2 Software Architecture 30
6.3 Software Interfacing: MATLAB/Simulink, LabView & Python 31
6.4 System Identification and Analysis 34
7. CRITICAL ANALYSIS OF PROJECT SHORTCOMINGS AND ENGINEERING CHALLENGES 35
7.1 Cancellation of Infrared (IR) Sensors 35
7.2 Actuator Failure and Degradation: From Encoder to TT Motors 35
7.3 Inter-Software Fusion Challenges: MATLAB/Simulink and LabVIEW 36
8. RESULTS AND PERFORMANCE ANALYSIS 36
9. CONCLUSION AND FUTURE RECOMMENDATIONS 37
10. REFERENCES 38
11. APPENDIX 42
11.1 Arduino Code 42
11.2 Python Code 50

LIST OF FIGURES

Figure 1: Project Robot 6
Figure 2: Robot Dynamics 8
Figure 3: MPU6050 15
Figure 4: HC SR-04 Ultrasonic Sensor 16
Figure 5: L298N Dual H-Bridge Motor Driver 17
Figure 6: Arduino Mega 2560 18
Figure 7: Motor and Wheel 19

Figure 8: Circuit Schematic.....20
Figure 9: Flowchart of a Simple Balancing Robot.....28
Figure 10: Zeigler-Nichols PID Tuning.....29
Figure 11: Simulink Modelling.....29
Figure 12: System Response 30
Figure 13: MATLAB Interfaced Code 32
Figure 14: Simulink Interfaced Model 32
Figure 15: LabView Interfaced VI Model.....33
Figure 16: Python Interface Visualization 34

LIST OF TABLES

Table 1: Performance Metrics of the Final System 36

ABSTRACT

This report investigates the design, mathematical modelling, and real-time control of a Two-Wheeled Self-Balancing Robot (TWSBR). Functioning as an inherently unstable, underactuated inverted pendulum, the system serves as a rigorous terrestrial analogue for complex aerospace attitude control challenges. The governing equations of motion are derived using the Euler-Lagrange formalism to systematically account for nonholonomic constraints and coupled multi-body dynamics. Control is executed via a hierarchical Proportional-Integral-Derivative (PID) architecture embedded on an Arduino Mega 2560 microcontroller, utilizing a complementary filter for sensor fusion of MPU6050 Inertial Measurement Unit (IMU) data to ensure robust attitude estimation. Furthermore, the system integrates ultrasonic proximity sensing to enable reactive autonomous obstacle avoidance. Performance is validated through Hardware-in-the-Loop (HIL) simulation and real-time telemetry analysis using MATLAB/Simulink and LabVIEW, confirming the efficacy of the proposed control strategy in maintaining dynamic equilibrium and executing controlled navigation.

INTRODUCTION

The rigorous study of dynamical systems that lack inherent stability constitutes a cornerstone of modern control engineering and aerospace mechanics. The Two-Wheeled Self-Balancing Robot (TWSBR) represents a canonical problem within this domain, offering a tangible, physical manifestation of the classic inverted pendulum on a cart. Unlike a conventional pendulum, which naturally seeks a stable equilibrium point at the nadir (hanging downwards) due to gravitational restoring forces, the inverted pendulum possesses a saddle-point equilibrium at the zenith. This position is statically unstable; any infinitesimal perturbation from the vertical results in a gravitational torque that accelerates the mass away from the equilibrium, necessitating continuous, active control intervention to maintain upright stability.

1.1 The Inverted Pendulum as an Aerospace Analog

The engineering relevance of the TWSBR extends far beyond terrestrial robotics; it is deeply rooted in the fundamental physics of flight and spaceflight. In the context of Aerospace Engineering, the dynamics of the TWSBR effectively mirror the instability encountered in rocketry, particularly during the initial boost phase. A liquid-propellant launch vehicle ascending vertically is essentially a massive, inverted pendulum balanced on a thrust vector. If the center of pressure (CP) lies ahead of the center of gravity (CG) – which is often the case during the transonic regime or due to payload fairing geometry – the rocket is aerodynamically unstable. Without an active control loop adjusting the gimbal angle of the engines (Thrust Vector Control or TVC), the rocket would tumble, leading to structural failure. The control logic required to balance the TWSBR by moving its wheels is mathematically homologous to the logic required to balance a rocket by gimbaling its engines.

Similarly, the problem of stabilizing the TWSBR is analogous to attitude control in satellites and spacecraft. Spacecraft often employ momentum wheels or reaction wheels to maintain orientation without expending propellant. The TWSBR maintains its pitch angle (θ) by accelerating its wheels to generate a reaction torque on the chassis, governed by the

conservation of angular momentum. This interaction mimics the physics of a Reaction Wheel Assembly (RWA) used to reject disturbance torques in orbit. Consequently, the development of control algorithms for the TWSBR – specifically those involving sensor fusion for Attitude, Heading, and Reference Systems (AHRS) and high-frequency feedback loops – provides a direct pedagogical and experimental pathway to understanding flight dynamics and avionics.

1.2 Problem Definition and Classification

The TWSBR is classified as a multi-variable, high-order, non-linear, underactuated system. The underactuated classification arises because the system possesses fewer control inputs (two motor torques) than degrees of freedom (tilt angle θ, heading angle ψ, and planar position x, y). This underactuation implies that the system cannot directly command an arbitrary state; for example, the robot cannot simply move forward without first manipulating its pitch angle to generate a gravitational component of force. This leads to non-minimum phase behaviour – where the system must initially move in the direction opposite to the desired target to induce the necessary lean – adding significant complexity to the control topology.

1.3 Objectives

The primary objective of this thesis is the comprehensive development, modelling, and implementation of a mobile TWSBR capable of simultaneous vertical stabilization and autonomous navigation in a 2D plane. This involves several critical sub-objectives:

1. **Dynamic Modelling:** To derive a high-fidelity mathematical model using Lagrangian mechanics to predict system behaviour and identify coupling terms.
2. **State Estimation:** To implement robust sensor fusion algorithms (Complementary Filter) to determine the robot's attitude in real-time, overcoming the noise and drift inherent in low-cost MEMS sensors.
3. **Control Synthesis:** To design a hierarchical control structure that balances the fast dynamics of the pendulum (inner loop) with the slower dynamics of navigation (outer loop).
4. **Autonomous Navigation:** To integrate exteroceptive sensors (ultrasonic) to detect and avoid obstacles without compromising vertical stability.
5. **Validation and Failure Analysis:** To validate the system performance through empirical testing and to rigorously analyse the engineering shortcomings, specifically regarding sensor integration failures and actuator degradation.

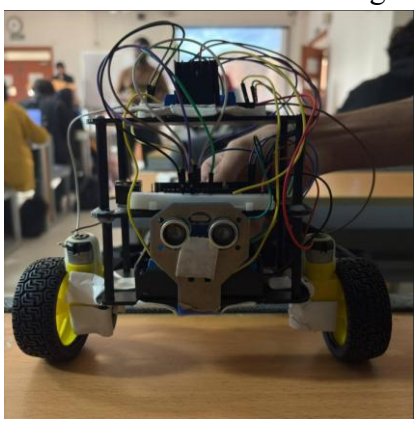


Figure 1: Project Robot

2. THEORETICAL FRAMEWORK AND MATHEMATICAL MODELLING

To design an effective controller, one must first establish a rigorous mathematical description of the physical plant. While Newton-Euler methods can be used, the Euler-Lagrange formalism is preferred for complex multi-body systems like the TWSBR as it avoids the need to explicitly solve for internal reaction forces between the wheels and the chassis, focusing instead on the system's energy properties.

2.1 Coordinate Systems and Kinematics

The system is defined using a set of generalized coordinates$q = [x, y, \theta, \psi]^T$, where:

- (x, y) represents the Cartesian position of the midpoint of the wheel axis in the inertial frame.
- θ represents the pitch angle of the intermediate body (pendulum) relative to the vertical axis (where $\theta = 0$ is the upright equilibrium).
- ψ represents the yaw (heading) angle of the robot in the global frame.

The kinematic constraints are governed by the assumption of pure rolling without slipping. For a wheel of radius r_w and a track width d (distance between wheel centers), the linear velocity v and angular yaw rate $\dot{\psi}$ are related to the angular velocities of the right ($\dot{\phi}_R$) and left ($\dot{\phi}_L$) wheels. The linear velocity of the robot body is given by the average of the wheel velocities:

$$v = \dot{x_{body}} = \frac{r_w}{2}(\dot{\phi}_R + \dot{\phi}_L)$$

The angular yaw rate is determined by the differential velocity of the wheels:

$$\dot{\psi} = \frac{r_w}{d}(\dot{\phi}_R - \dot{\phi}_L)$$

These equations represent nonholonomic constraints. A nonholonomic constraint is defined as a constraint involving velocities that cannot be integrated to yield a relationship between positions alone. Mathematically, this is expressed as an equation of the form $f(q, \dot{q}, t) = 0$ that is not integrable. Physically, this implies that the robot possesses three degrees of freedom in positioning (x, y, ψ) but only two control inputs (ϕ_R, ϕ_L). The robot cannot translate sideways (along the wheel axis) instantly; it must execute a manoeuvring turn to reach a specific (x, y) coordinate. This constraint is fundamental to the path planning logic required for obstacle avoidance and distinguishes wheeled mobile robots from holonomic systems like omnidirectional platforms.

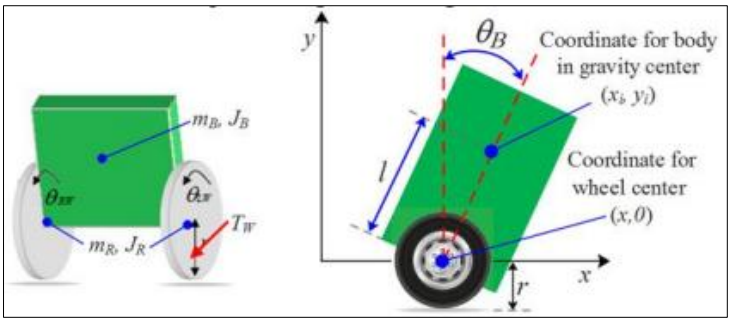


Figure 2: Robot Dynamics

2.2 Lagrangian Dynamics Derivation

The dynamics are derived using the Lagrangian $\mathcal{L}$, defined as the difference between the total kinetic energy ($\mathcal{T}$) and the total potential energy ($\mathcal{V}$) of the system:

$$\mathcal{L}(q, \dot{q}) = \mathcal{T}(q, \dot{q}) - \mathcal{V}(q)$$

2.2.1 Kinetic Energy Analysis ($\mathcal{T}$)

The total kinetic energy of the system is the sum of the translational and rotational kinetic energies of its constituent rigid bodies: the main chassis (pendulum) and the two wheels.

1. Chassis Kinetic Energy ($\mathcal{T}_{body}$):

The chassis has a mass M_b and a moment of inertia I_b about its center of mass (CoG). The CoG is located at a distance l from the wheel axis. The velocity of the CoG is a vector sum of the linear velocity of the wheel axis and the tangential velocity generated by the rotation of the pendulum. The translational velocity squared of the CoG, v_{CoG}^2, can be expressed using the law of cosines as:

$$v_{CoG}^2 = \dot{x}^2 + \left(l\dot{\theta}\right)^2 + 2\dot{x}\left(l\dot{\theta}\right)\cos(\theta)$$

Therefore, the chassis kinetic energy is:

$$\mathcal{T}_{body} = \frac{1}{2}M_b\left(\dot{x}^2 + 2\dot{x}l\dot{\theta}\cos(\theta) + l^2\dot{\theta}^2\right) + \frac{1}{2}I_b\dot{\theta}^2$$

This equation highlights a critical coupling term: $\dot{x}\dot{\theta}\cos(\theta)$. This term mathematically represents the interaction between the linear motion of the cart and the rotational motion of the pendulum. It is this specific coupling that enables the balancing mechanism; by controlling the linear acceleration $\ddot{x}$, we can influence the angular dynamics of θ.

2. Wheel Kinetic Energy ($\mathcal{T}_{wheels}$):

Each of the two wheels has a mass M_w, a radius r_w, and a moment of inertia I_w. The wheels contribute to kinetic energy in two ways: translating along the ground and rotating about their axles.

$$\mathcal{T}_{wheels} = 2 \times \left(\frac{1}{2}M_w\dot{x}^2 + \frac{1}{2}I_w\dot{\phi}^2\right)$$

Using the non-slip condition $\dot{x} = r_w\dot{\phi}$, we can substitute $\dot{\phi} = \dot{x}/r_w$:

$$\mathcal{T}_{wheels} = \left(M_w + \frac{I_w}{r_w^2}\right)\dot{x}^2$$

Summing these components gives the total system kinetic energy $\mathcal{T}$.

2.2.2 Potential Energy Analysis (V)

The potential energy of the system is determined by the vertical position of the centres of mass. Assuming the ground is the reference level ($h = 0$):

- **Wheels:** The center of mass of the wheels remains at a constant height r_w. Thus, the change in potential energy is zero ($\Delta\mathcal{V}_{wheels} = 0$).
- **Chassis:** The CoG of the chassis varies with the tilt angle θ. The height of the CoG is $h_{CoG} = r_w + l\cos(\theta)$.

Since the derivative of a constant ($M_b g r_w$) is zero, the relevant potential energy term is simply:

$$\mathcal{V} = M_b g l \cos(\theta)$$

It is crucial to note the sign convention. If $\theta = 0$ is upright, the potential energy is maximized. The system naturally tends towards the state of minimum potential energy (hanging down, $\theta = \pi$), confirming the inherent instability of the upright position.

2.3 Equations of Motion and Linearization

The equations of motion are obtained by applying the Euler-Lagrange differential equation:

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{q}_i}\right) - \frac{\partial \mathcal{L}}{\partial q_i} = Q_i$$

where q_i are the generalized coordinates (x, θ) and Q_i represents the generalized non-conservative forces acting on those coordinates (motor torque τ, friction B, external disturbances).

Applying this to the pitch angle θ and horizontal position x yields a set of two coupled, second-order, non-linear differential equations. For the pitch dynamics (the critical balancing axis), the equation takes the form:

$$(I_b + M_b l^2)\ddot{\theta} + M_b l \cos(\theta)\,\ddot{x} - M_b g l \sin(\theta) = -\tau$$

This equation reveals the fundamental balancing mechanism: to influence the angular acceleration $\ddot{\theta}$, the system can either apply a motor torque τ or induce a linear acceleration $\ddot{x}$. The term $M_b l \cos(\theta)\,\ddot{x}$ represents the inertial force or fictitious force acting on the CoG due to the acceleration of the base. By accelerating the wheels ($\ddot{x}$), a reaction torque is applied to the pendulum, counteracting the gravitational torque $M_b g l \sin(\theta)$.

To facilitate the design of a Linear Time-Invariant (LTI) controller such as PID, these non-linear equations must be linearized around the operating point. The operating point for a self-balancing robot is the unstable vertical equilibrium:

- $\theta \approx 0$
- $\dot{\theta} \approx 0$

Applying small-angle approximations:

- $\sin(\theta) \approx$
- $\cos(\theta) \approx 1$
- $\dot{\theta}^2 \approx 0$ (Coriolis/Centripetal terms negligible near equilibrium)

This linearization yields a State-Space representation of the form $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$, where the state vector is $\mathbf{x} = [\theta, \dot{\theta}, x, \dot{x}]^T$ and the input $\mathbf{u}$ is the motor voltage or duty cycle. The system matrix A contains positive eigenvalues, mathematically confirming that the open-loop system is unstable (poles in the right-half plane).

2.4 Transfer Function and State-Space

The transfer function relating the input force (F) to the pendulum angle ($\Phi(s)$) is derived as:

$$P_{pend}(s) = \frac{\Phi(s)}{U(s)} = \frac{\frac{ml}{q}s^2}{s^4 + \frac{b(I+ml^2)}{q}s^2 - \frac{(M+m)mgl}{q}}$$

Where q captures the mass and inertia properties. The presence of a pole in the right-half plane (positive real root in the denominator characteristic equation) mathematically confirms the system is unstable without feedback.

The State-Space representation $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$ uses the state vector $\mathbf{x} = [\theta, \dot{\theta}, x, \dot{x}]^T$. The system matrix A contains the linearized dynamics, and its eigenvalues determine stability.

$$\mathrm{A} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ \frac{M_t mgl}{p} & \frac{-(I+ml^2)b}{p} & 0 & 0 \\ 0 & 0 & 0 & 0 \\ \frac{-m^2gl^2}{p} & \frac{-mlb}{p} & 0 & 0 \end{bmatrix}$$

(where p is the determinant of the inertia matrix). This format is essential for advanced control methods like LQR.

The linearized state space formulation for this robot, relating the system's output and the voltage input is given by

$$\dot{\xi} = A\xi + Bu,$$
$$y = C\xi + Du,$$

$$\xi = [x \quad \dot{x} \quad \theta \quad \dot{\theta}]^T$$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & \alpha & \beta & -r\alpha \\ 0 & 0 & 0 & 1 \\ 0 & \gamma & \delta & -r\gamma \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & \alpha\varepsilon & 0 & \gamma\varepsilon \end{bmatrix}^T$$

where

$$\alpha = \frac{2\left(Rb - K_e K_m\right)\left(M_b L^2 + M_b r L + J_b\right)}{R\left(2\left(J_b J_w + J_w L^2 M_b + J_b M_w r^2 + L^2 M_b M_w r^2\right) + J_b M_b r^2\right)}$$

$$\beta = \frac{-L^2 {M_b}^2 g r^2}{J_b\left(2J_w + M_b r^2 + 2M_w r^2\right) + 2J_w L^2 M_b + 2L^2 M_b M_w r^2}$$

$$\gamma = \frac{-2\left(Rb - K_e K_m\right)\left(2J_w + M_b r^2 + 2M_w r^2 + L M_b r\right)}{Rr\left(2\left(J_b J_w + J_w L^2 M_b + J_b M_w r^2 + L^2 M_b M_w r^2\right) + J_b M_b r^2\right)}$$

$$\delta = \frac{L M_b g\left(2J_w + M_b r^2 + 2M_w r^2\right)}{2J_b J_w + 2J_w L^2 M_b + J_b M_b r^2 + 2J_b M_w r^2 + 2L^2 M_b M_w r^2}$$

$$\varepsilon = \frac{K_m r}{Rb - K_e K_m}$$

The final transfer function of the plant is given by

$$\frac{\theta_p}{V_s}(s) = \frac{\frac{K_t}{R_a}\left(m_p\, l_g\, r_w\right)}{\left[-\left(J_p - m_p\, l_g^2\right)\left[\frac{J_m}{G_r} + J_w + m_p r_w^2 + m_w r_w^2 + m_p l_g r_w\right]\right] s^2 - \left[\left(J_p - m_p\, l_g^2\right)\left[\frac{K_t\, K_e}{R_a\, G_r} + \frac{b}{G_r}\right]\right] s}$$

3. PROJECT WORKFLOW AND METHODOLOGY

The development of the Two-Wheeled Self-Balancing Robot (TWSBR) was accomplished through a structured, five-phased engineering lifecycle. This methodology was specifically designed to bridge the gap between theoretical Lagrangian dynamics and the practicalities of real-time embedded control, ensuring that the final prototype achieved high-fidelity stabilization and navigational autonomy.

3.1 Phase 1: Project Proposal and Mathematical Planning

The initial phase involved the definition of the control problem and the derivation of the system's mathematical foundation.

- Problem Definition: The system was classified as a multi-variable, high-order, non-linear, and underactuated plant. It was established as a terrestrial analogy to rocket thrust vector control and satellite reaction wheel assemblies.
- Analytical Modelling: The governing equations of motion were derived using the Euler-Lagrange formalism. This involved a detailed kinetic energy analysis (T) – accounting for chassis translation, chassis rotation about the Center of Gravity (CoG), and wheel

dynamics – and a potential energy analysis (V) relative to the unstable zenith equilibrium.

- Linearization and State-Space: To facilitate Linear Time-Invariant (LTI) control design, the non-linear equations were linearized using small-angle approximations around the operating point ($\theta \approx 0$). This resulted in a state-space representation ($x = Ax + Bu$) and plant transfer functions essential for analysing open-loop instability and designing the subsequent PID architecture.

3.2 Phase 2: Component Acquisition and Empirical Prototyping

This phase focused on selecting instrumentation based on physical sensor characteristics and developing initial simulation models.

- Instrumentation Selection: Components were selected based on their spectral characteristics. The MPU6050 MEMS IMU was chosen for its 6-DOF sensing, despite known spectral deficiencies such as high-frequency accelerometer noise and low-frequency gyroscope bias drift. Actuators were initially selected as JGA25 encoder motors for velocity feedback, later transitioning to high-torque TT motors due to gearbox reliability requirements.

- Simulink Plant Modelling: A high-fidelity Simulink model was constructed based on the derived transfer functions. The plant was first simulated in an open-loop configuration to confirm the presence of right-half plane poles, mathematically validating the inherent instability of the pendulum prior to controller integration.

3.3 Phase 3: System Integration and Development

The integration phase realized the Hardware-in-the-Loop (HIL) design through meticulous circuit and software fusion.

- Hardware Architecture and EMI Isolation: In accordance with circuit design best practices, the system utilized a split power distribution architecture. To mitigate Electromagnetic Interference (EMI) from DC motor brush arcing, the high-current motor power rails (11.1V) were physically isolated from the sensitive logic rails (regulated 5V), with all ground planes tied to a common reference potential to ensure signal integrity.

- Deterministic Software Architecture: The firmware was engineered around a Timer 3 Interrupt Service Routine (ISR) configured for a 200Hz (5ms) execution frequency. This ensured deterministic timing for the PID loop ($T_s = 5ms$), independent of non-pre-emptive tasks such as ultrasonic sonar reading, which was down sampled to 20Hz (every 50ms) to prevent CPU blocking.

- Sensor Fusion (Complementary Filter): A fusion algorithm was implemented to weight the integrated gyroscope data at 99.5% for short-term precision and accelerometer data at 0.5% for long-term gravity reference, overcoming the integration drift that otherwise causes unbounded tilt error.

3.4 Phase 4: Control Synthesis, Tuning, and Interfacing

This phase involved the final optimization of the system performance and the establishment of high-level software telemetry.

- Hierarchical PID Tuning: The controller was tuned using a heuristic Ziegler-Nichols approach. K_p was adjusted as a virtual spring for restoring torque, K_d as a virtual damper to dissipate energy and prevent oscillations, and Ki to eliminate steady-state leaning caused by mechanical CoG offsets.
- Multi-Platform Software Fusion: Real-time telemetry was established via a UART interface at 115200 baud. Interfacing was achieved through three distinct methods to facilitate data acquisition and monitoring:
    1. Simulink External Mode: For real-time state telemetry and gain tuning using External Mode.
    2. LabVIEW Virtual Instruments (VI): For graphical front-end monitoring and digital instrumentation design.
    3. Python Telemetry Dashboard: Developed to prevent blocking I/O latency through multi-threaded data acquisition and circular buffering (deque) in VS Code.

3.5 Phase 5: Performance Analysis and Refinement

The final phase prioritized the evaluation of system robustness and the documentation of engineering challenges.

- Stability Metrics: The system's response was analysed based on Rise Time (t_r), Overshoot (M_p), and Settling Time (t_s), with the robot successfully settling within 2.5 seconds after a disturbance.
- Obstacle Avoidance Logic: Reactive autonomous behaviour was refined to trigger a 'Stop-and-Turn' manoeuvre within 15cm of an obstruction, successfully integrating exteroceptive sensing with the stabilization loop.
- Troubleshooting and System Identification: Discrepancies between the Lagrangian model and real-world behaviour were identified, specifically analysing the impact of gearbox backlash ($> 5°$) and motor dead-zones on the system's limit-cycle oscillations, as well as the descope of IR line-following sensors due to ADC resource conflicts.

4. AEROSPACE INSTRUMENTATION AND AVIONICS ARCHITECTURE

In the aerospace domain, the subsystem responsible for determining the orientation and position of a vehicle is referred to as the Navigation, Guidance, and Control (GNC) system. For the TWSBR, precise determination of the pitch angle θ and pitch rate $\dot{\theta}$ is the single most critical factor for stability. The primary avionics component employed is the MPU6050, a Micro-Electro-Mechanical System (MEMS) 6-Degree-of-Freedom (6-DOF) Inertial Measurement Unit (IMU).

4.1 Sensor Physics and Spectral Characteristics

The MPU6050 integrates a 3-axis gyroscope and a 3-axis accelerometer on a single silicon die. Understanding the physics and error characteristics of these sensors is essential for designing the sensor fusion algorithm.

4.1.1 MEMS Accelerometer Principles

The accelerometer measures proper acceleration, which includes both kinematic acceleration ($\ddot{x}$) and the reaction force against gravity (g). Internally, it consists of a microscopic proof mass suspended by polysilicon springs between fixed capacitive plates. When the sensor tilts, the component of gravity acting along the sensitive axis deflects the proof mass, changing the differential capacitance.

- **Utility:** It provides an absolute reference for the down vector (gravity).
- **Limitation:** It is extremely sensitive to high-frequency noise. Motor vibrations, chassis resonance, and sudden linear accelerations ($\ddot{x}$) are indistinguishable from tilt in the short term. If the robot accelerates forward quickly, the accelerometer will register a vector sum of gravity and linear acceleration, reporting an erroneous tilt angle. This phenomenon makes raw accelerometer data unusable for dynamic balancing.

4.1.2 MEMS Gyroscope Principles

The gyroscope measures angular velocity (ω) using the Coriolis effect. A vibrating structure (proof mass) within the sensor is subjected to a Coriolis force when the sensor rotates, which is detected capacitively.

- **Utility:** It provides a clean, high-bandwidth signal that accurately reflects the rate of rotation ($\dot{\theta}$) and is insensitive to linear acceleration.
- **Limitation:** It suffers from low-frequency errors, specifically bias drift and 'random walk'. To obtain the angle θ, the angular velocity ω must be integrated ($\theta = \int \omega dt$). Any small DC bias in the raw signal is integrated over time, causing the calculated angle to drift unboundedly, even if the robot is stationary. Within minutes, the robot would think up is sideways and crash.

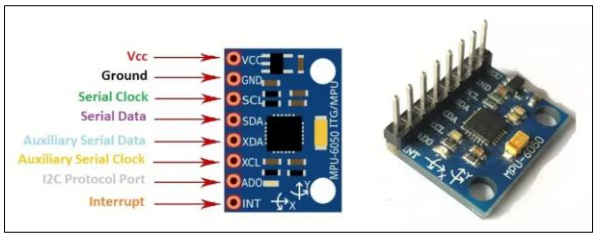


Figure 3: MPU6050

4.2 Sensor Fusion: The Complementary Filter

To reconstruct a robust and accurate estimate of the pitch angle θ, sensor fusion is required to combine the long-term stability of the accelerometer with the short-term precision of the gyroscope. While aerospace applications often employ Extended Kalman Filters (EKF) for optimal estimation, the TWSBR typically utilizes a Complementary Filter due to its computational efficiency on 8-bit microcontrollers like the Arduino Mega 2560.

The Complementary Filter operates in the frequency domain. It applies a Low-Pass Filter (LPF) to the accelerometer data (to reject high-frequency vibration) and a High-Pass Filter (HPF) to the integrated gyroscope data (to reject low-frequency drift). The fusion equation is given by:

$$\theta_k = \alpha\left(\theta_{k-1} + \omega_{gyro} \cdot \Delta t\right) + (1 - \alpha)\theta_{accel}$$

Where:

- θ_k is the fused angle estimate at time step k.
- ω_{gyro} is the raw gyroscope rate.
- Δt is the sampling loop time (critical for integration).
- θ_{accel} is the angle calculated from the accelerometer gravity vector ($\arctan(a_x/a_z)$).
- α is the filter coefficient, typically close to unity (e.g., 0.98).

This equation implies that 98% of the new angle estimate comes from the gyroscope integration (preserving dynamic response), while 2% comes from the accelerometer to slowly correct the gyroscope's drift back to the true vertical. This fusion provides the low-latency, low-noise θ signal necessary for the Derivative (D) term in the PID controller to function without amplifying noise.

4.3 Environmental Sensing: Ultrasonic Range Finding

For obstacle avoidance, the robot employs the HC-SR04 ultrasonic sensor. This sensor operates on the principle of Time-of-Flight (ToF). It emits an 8-cycle burst of ultrasonic sound at 40 kHz and listens for the echo reflected from an obstacle.

The distance D is calculated as:

$$D = \frac{T_{flight} \times V_{sound}}{2}$$

where $V_{sound} \approx 343$ m/s at sea level. The sensor has a detection range of 2 cm to 400 cm. However, in the context of a balancing robot, the sensor's mounting is critical. Since the robot tilts to move, the ultrasonic sensor (fixed to the chassis) also tilts. If the robot leans forward significantly to accelerate, the sensor might point at the floor, generating false positive obstacle

detections. This geometric coupling between attitude and sensing must be accounted for in the navigation logic.

Figure 4: HC SR-04 Ultrasonic Sensor

4.4 Driving System: L298N Dual H-Bridge Motor Driver

The L298N is a high-voltage, high-current dual full-bridge driver designed to accept standard TTL logic levels and drive inductive loads such as relays, solenoids, DC and stepping motors. It serves as the power interface between the low-voltage logic (5V) of the microcontroller and the high-voltage (12V) requirements of the motors.

The L298N utilizes an H-Bridge circuit configuration, which allows voltage to be applied across a load in either direction. The 'H' structure is formed by four switching elements (typically BJT transistors in the L298N).

- **Forward Motion:** Closing switches S1 and S4 (High-Side Left and Low-Side Right) allows current to flow through the motor from left to right.
- **Reverse Motion:** Closing switches S2 and S3 (High-Side Right and Low-Side Left) reverses the current flow.
- **Speed Control:** The Enable pins (ENA/ENB) are driven by a Pulse Width Modulation (PWM) signal. The L298N chops the output voltage at the PWM frequency, effectively controlling the average voltage and torque delivered to the motor.

Technical Specifications:

- Operating Voltage: Up to 46V.
- DC Current: 2A per channel (4A total).
- Saturation Voltage (V_{CEsat}): The internal Darlington transistors result in a significant voltage drop (approx. 1.8V to 3.2V) across the driver, which must be accounted for in the power budget. It dissipates this energy as heat, necessitating a heatsink.

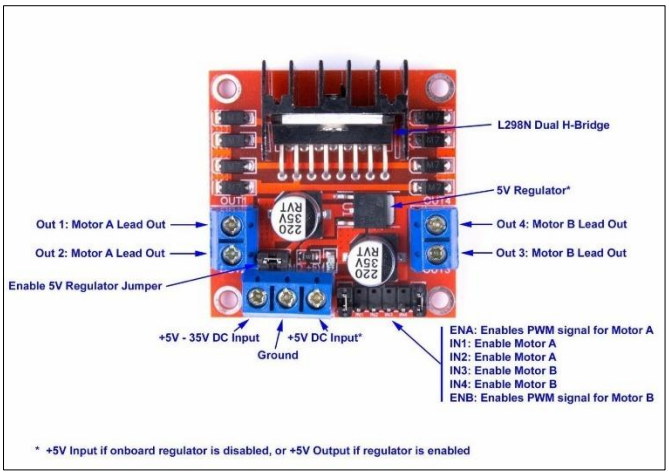


Figure 5: L298N Dual H-Bridge Motor Driver

4.5 Processing Unit: Arduino Mega2560

The physical implementation of the TWSBR requires a meticulous integration of power electronics, sensing elements, and computational hardware. The nervous system of the robot is built around the Arduino Mega 2560 microcontroller, which interfaces with high-current drivers and precision sensors.

The Arduino Mega 2560 was selected as the central processing unit due to its superior I/O capabilities and memory compared to standard microcontroller boards. Based on the ATmega2560 datasheet, the board operates at 16 MHz and provides 54 digital I/O pins, 16 analogue inputs, and 4 hardware UARTs.

The mega variation was selected based on the following criteria:

- Interrupt Capability: The ATmega2560 provides six external interrupt pins (2, 3, 18, 19, 20, 21), which are crucial for reading quadrature encoder signals without polling, allowing for precise odometry and velocity measurement.2
- Timer Resources: The board features multiple hardware timers (Timer 0, 1, 2, 3, 4, 5). This project specifically utilizes Timer 3 to generate a precise 200 Hz hardware interrupt for the PID control loop. This ensures deterministic timing ($T_s = 5ms$) independent of the main loop execution time, a requirement for stable digital control.
- Memory: With 256 KB of Flash memory and 8 KB of SRAM, the Mega can accommodate the substantial overhead of floating-point PID calculations and sensor fusion libraries.

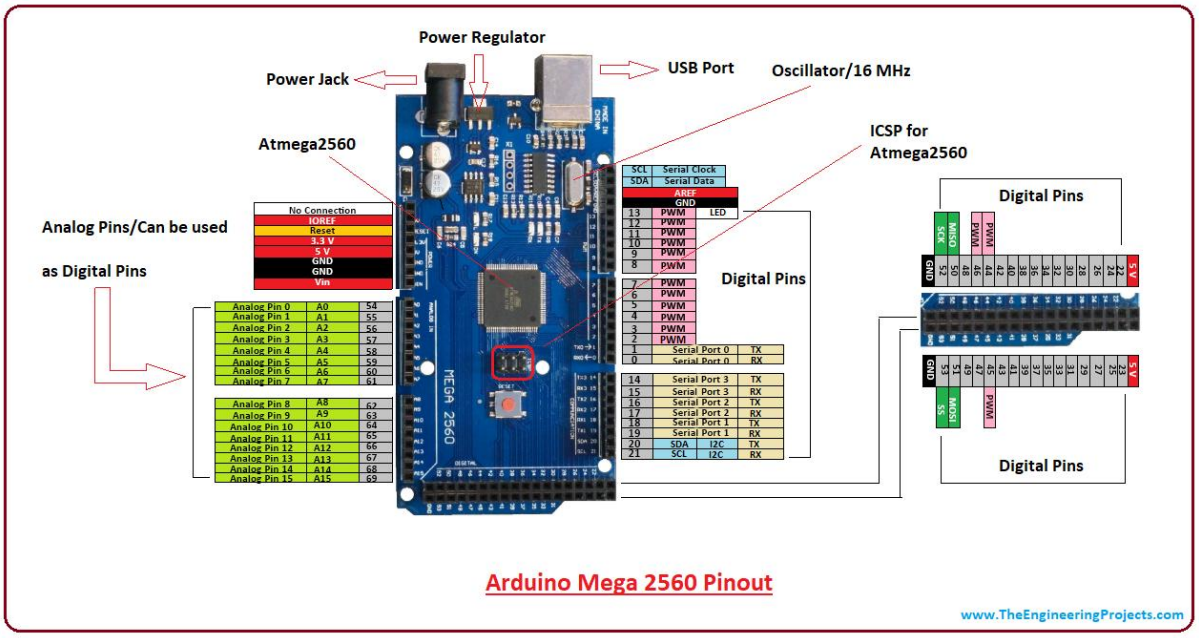


Figure 6: Arduino Mega 2560

4.6 Actuation System: TT Geared DC Motor and Wheel Assembly

The actuation subsystem serves as the physical interface between the control algorithm and the environment. For the TWSBR, this system must convert the electrical control signals (u) generated by the PID controller into the mechanical torque (τ) required to induce angular acceleration of the pendulum body. The robot utilizes a pair of TT style DC Geared Motors coupled with high-traction rubber wheels.

4.6.1 Electromechanical Actuator: The TT Gear Motor

The core actuator is a Yellow TT (Double Shaft) Gear Motor, a ubiquitous component in educational robotics chosen for its wide availability and high torque-to-weight ratio.

- **Motor Topology:** The unit consists of a small form-factor (130-size) Permanent Magnet DC (PMDC) motor. It operates on the principle of Lorentz force, where current flowing through the armature windings interacts with the permanent magnetic field of the stator to generate rotational torque.
- **Transmission System:** To effectively balance the robot, high torque is prioritized over high speed. The motor shaft spins at high RPM with low torque, which is unsuitable for direct driving the wheels. Therefore, an integrated gearbox is employed.
    - <u>Gear Train:</u> The unit houses a multi-stage spur gear train constructed from injection-moulded nylon/plastic.
    - <u>Reduction Ratio:</u> The specific model employed features a gear reduction ratio of 1:48. This signifies that for every 48 revolutions of the motor armature, the output shaft completes one full revolution, mechanically amplifying the torque by a factor of approximately 48 (minus efficiency losses).

- **Technical Specifications:**
    - Operating Voltage: 3V to 6V DC (Nominal).
    - No-Load Speed: Approximately 200 RPM at 6V supply.
    - Stall Torque: ≈ 0.8 kg · cm (at 6V).
    - No-Load Current: ≈ 200 mA.
    - Stall Current: ≈ 1.5 A.

4.6.2 Traction Interface: The 65mm Wheel

The transfer of torque from the motor to the ground is mediated by the wheel assembly. The characteristics of the wheel dictate the friction forces available to accelerate the cart base ($M\ddot{x}$).

- **Geometry:** The wheel has an outer diameter of 65 mm and a width of 26 mm. The radius ($r_w = 32.5$ mm) is a critical parameter in the kinematic equations ($v = \omega r_w$), directly influencing the linear resolution of the robot's movement.
- **Material Composition:** The central rim is rigid plastic, while the tire is composed of soft, treaded rubber.
- **Friction Dynamics:** The rubber tire provides a high coefficient of static friction (μ_s) with typical flooring surfaces. This is essential for the TWSBR because the balancing logic often commands rapid, high-torque reversals. Low traction would result in wheel slip (burnout), decoupling the motor angle (ϕ) from the cart position (x) and causing the Lagrangian control model – which assumes pure rolling without slipping – to fail.

Figure 7: Motor and Wheel

4.7 System Design

A critical design requirement for aerospace-grade reliability is Electromagnetic Interference (EMI) Isolation. DC motors generate significant electrical noise (brush arcing) and inductive voltage spikes during switching. If the Arduino and IMU share the same unfiltered power rails

as the motors, this noise can couple into the sensitive I2C lines of the MPU6050, causing data corruption or sensor freezing.

To mitigate this, the system employs a split power distribution architecture:

- Motor Power: High-current Li-ion batteries (7.4V - 11.1V) directly power the L298N Vs terminal.
- Logic Power: A regulated 5V supply powers the Arduino and sensors.
- Common Ground: Crucially, the ground planes of the motor supply and logic supply are tied together to provide a common reference potential for the control signals.

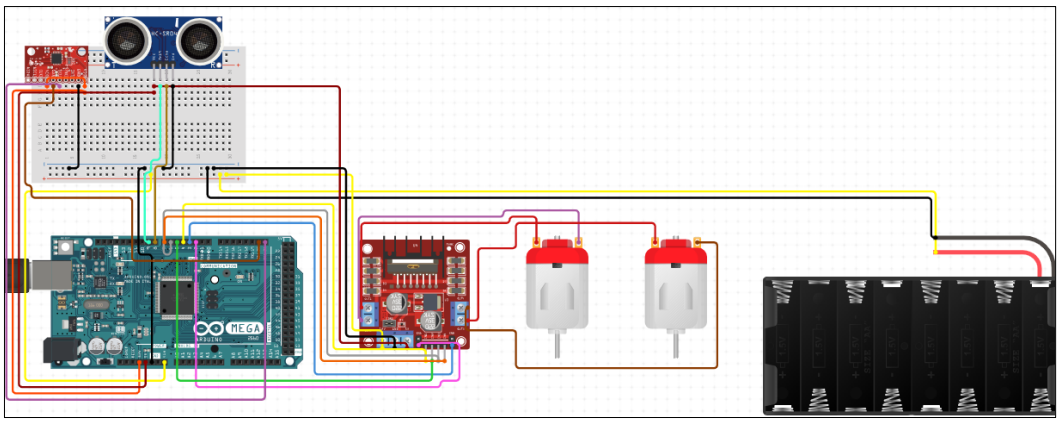


Figure 8: Circuit Schematic

4.7.1 Circuit and Code Breakdown

A. The Motor Driver (L298N)

The L298N acts as a high-current amplifier. The Arduino sends weak 5V logic signals; the L298N boosts them to drive the motors using battery power.

Component Pin	Arduino Pin	Code Variable	Logic & Explanation
ENA	Pin 6	ENA	PWM (Pulse Width Modulation). Pin 6 is a PWM-capable pin on the Mega. It sends a pulsing signal (0-255) to control the speed of the Left Motor. 0 = Stop, 255 = Full Speed.

IN1	Pin 24	IN1	Digital Logic. Controls Left Motor direction. If IN1=LOW and IN2=HIGH, the motor spins Forward.
IN2	Pin 25	IN2	Digital Logic. Inverse of IN1. If IN1 is HIGH, this must be LOW to reverse direction.
IN3	Pin 26	IN3	Digital Logic. Controls Right Motor direction. Same logic as IN1/IN2 but for the right side.
IN4	Pin 27	IN4	Digital Logic. Controls Right Motor direction.
ENB	Pin 7	ENB	PWM. Pin 7 is also PWM-capable. It controls the speed of the Right Motor independently.

Logic in Code: In `driveMotors()`, `analogWrite(ENA, PWM)` is written to set speed and `digitalWrite(IN1, ...)` to set direction.

B. Gyroscope and Accelerometer (MPU6050)

The MPU6050 measures gravity and rotation to tell the robot if it is falling.

Component Pin	Arduino Pin	Code Variable	Logic & Explanation

VCC	5V	N/A	Powers the sensor.
GND	GND	N/A	Common ground reference.
SDA	Pin 20	Wire Library	Serial Data. Pin 20 is the dedicated SDA pin on the Arduino Mega for I2C communication. It carries the actual data bits (Angle/Gyro readings).
SCL	Pin 21	Wire Library	Serial Clock. Pin 21 is the SCL pin. It pulses to synchronize the data transfer, so bits don't get lost.
INT	Not Used	N/A	The code uses a Timer Interrupt (internal clock) rather than the external MPU interrupt pin for stability.

Logic in Code: The `Wire.h` library automatically knows to look at Pins 20 and 21 on the Mega when `Wire.begin()` is called.

C. HC-SR04 Ultrasonic

This sensor calculates distance by measuring how long sound takes to bounce back.

Component Pin	Arduino Pin	Code Variable	Logic & Explanation

VCC	5V	N/A	Powers the sensor.
GND	GND	N/A	Ground.
Trig	Pin 22	TRIG_PIN	Output. The Arduino sends a 10-microsecond pulse here to tell when to send the next pulse
Echo	Pin 23	ECHO_PIN	Input. The sensor holds this pin HIGH for exactly as long as the sound wave is in the air. The Arduino measures the length of this HIGH signal.

D. Constants

i. `#include <Wire.h>`

Imports the library needed to talk to the MPU6050 sensor via the SDA/SCL pins.

ii. `const bool DEBUG_MODE = false;`

A safety flag. If set to true, the motors are disabled, and the robot just prints sensor data to the Serial Monitor. Useful for testing if the MPU is working without the robot running away.

iii. PID Coefficients

```
float Kp = 75.0;
float Ki = 1.2;
float Kd = 2.0;
```

- Kp (75.0): If the robot leans 1 degree, motors respond with power 15.

- Ki (1.2): If the robot has been leaning for a while, this slowly adds power to push it back.
- Kd (2.0): If the robot is falling fast, this reacts to slow it down.

iv. `float targetAngle = 1.0;`

Mechanical Zero.

v. `const int ENA = 10, IN1 = 8...; // (Pin Mapping)`

Defines the English names for the pin numbers mapped in Part 1.

vi. `volatile bool runLoop = false;`

Volatile tells the Arduino "This variable might change unexpectedly (by an interrupt), so check it fresh every time." This flag controls the heartbeat of the robot.

E. The Setup (Initialization)

i. `void setup() {`

```
Serial.begin(115200);
Wire.begin();
Wire.setClock(400000);
```

Starts USB communication.

Starts I2C (Sensor) bus. `setClock(400000)` speeds up communication to 400kHz (Fast Mode) so we don't lag reading the sensor.

ii. `// Pins`

`pinMode(ENA, OUTPUT);`

Configures the pins. Motor pins are OUTPUT (sending voltage). Echo pin is INPUT (reading voltage).

iii. `// Timer 3 Interrupt`

`TCCR3A = 0; ... OCR3A = 1249; ... TIMSK3 |= (1 << OCIE3A);`

This complex block creates a Hardware Interrupt.

It forces the Arduino to pause whatever it is doing exactly every 5 milliseconds (200 times a second).

This guarantees the PID loop runs at a perfect, steady rhythm, which is crucial for stability.

iv. `ISR(TIMER3_COMPA_vect) { runLoop = true; }`

This is the function the timer calls every 5ms. It simply sets runLoop to true, acting like a go signal for the main loop.

F. The Main Loop

i.
```
void loop() {
  if (runLoop) {
    runLoop = false;
```

The code waits here until the 5ms timer. This ensures the robot doesn't run too fast or too slow.

ii. `readMPU();`

Calls the function to get the latest tilt angle.

iii.
```
static int div = 0;
if (div++ > 10) { ... readSensors(); }
```

The robot needs to balance every 5ms, but the ultrasonic sensor is slow. So through this logic the ultrasonic sensor is checked every 10th loop (every 50ms).

iv. `if (abs(currentAngle) > 40) { stopMotors(); ... }`

If the robot falls past 40 degrees, it can't recover. The power is cut so it doesn't flail around on the floor.

v.
```
float error = targetAngle - currentAngle;
errorSum += error * 0.005;
float dInput = (currentAngle - lastAngle) / 0.005;
float pidOut = (Kp * error) + (Ki * errorSum) - (Kd * dInput);
```

<u>The PID Math.</u>

- error: How far is the robot from upright?
- errorSum: Accumulates error to fix drift (Integral).
- dInput: How fast is the angle changing? (Derivative).
- pidOut: The final result – how much power to send to the motors.

vi. `float leftSpeed = pidOut + turnOffset;`

```
float rightSpeed = pidOut - turnOffset;
driveMotors(leftSpeed, rightSpeed);
```

Mixing Steering.

- We take the Balance Power (pidOut).
- We add turnOffset to the Left and subtract from the Right.
- If turnOffset is 0, both move equally (Straight). If turnOffset is 50, it spins.

G. Helper Functions

i. `readMPU()`

Sends a request to the MPU6050 address (0x68) and asks for 14 bytes of data.

atan2(ax, az): Uses basic trigonometry on the Accelerometer X and Z values to calculate which way is down (gravity).

Complementary Filter.

- currentAngle = 0.99 * (Gyro) + 0.01 * (Accel)
- It trusts the Gyroscope (smooth) for 99% of the signal and uses the Accelerometer (noisy) for 1% just to keep the Gyro from drifting.

ii. `readSensors()`

Sends the "Ping" to the Ultrasonic sensor.

Listens for the echo and calculates distance in centimeters.

Logic Matrix.

- If distance < 15cm: Set turnOffset = 60. This forces the robot to spin in place to avoid the obstacle.
- If Left IR sees Line: Set turnOffset = -20. Turn left gently.

iii. `driveMotors()`

constrain: Ensures that power more than 255 is not sent (which is impossible).

Deadzone Fix: Motors have friction; they won't move at power 10. This code forces them to jump to at least power 40 if they are supposed to move, ensuring a snappy response.

Writes the voltages to the pins defined in Part 1.

iv. `calibrateGyro()`

Runs once at startup. It reads the gyro 500 times while the robot is still, calculates the average error (drift), and stores it to subtract from future readings. This ensures the robot knows what still position is.

4.8 System Parameters

The following parameters are defined for the robot, for use in further calculations and modelling and simulation. Some of these values have been precisely calculated, while the others are an approximation based off multiple credible sources.

Chassis dimensions (L x W x H - from the center of motor shaft): 13cm x 6cm x 12.5cm

Mass of empty chassis: 100g

Mass of Arduino Mega 2560: 37g

Mass of battery pack: 150g

Mass of motors: 61g

Dimension of wheel: 65mm diameter, 2.7cm wheelbase

Mass of all other components (sensors, motor driver, breadboard strips, wires): 50g

Total mass: ≈ 400g

5. CONTROL SYSTEM SYNTHESIS

The control strategy adopted for the TWSBR is a Hierarchical (Cascaded) PID Architecture. This approach is favoured in aerospace applications (e.g., autopilot design) because it decouples the fast dynamics of stabilization from the slower dynamics of navigation.

5.1 The Inner Loop: Attitude Stabilization

The inner loop is the survival loop. It operates at a high frequency (ideally ≥ 100 Hz, implemented at 200 Hz in this project via Timer 3 Interrupts) and has the sole responsibility of keeping the robot upright.

- Input: The error between the desired pitch angle and the actual pitch angle: $e_\theta(t) = \theta_{ref}(t) - \theta_{measured}(t)$.
- Controller: A PID algorithm calculates the corrective motor command $u(t)$:

$$u(t) = K_p e_\theta(t) + K_i \int e_\theta(t)dt + K_d \frac{de_\theta(t)}{dt}$$

Physics of Terms:

- Proportional (K_p): Acts as a virtual spring. It generates a restoring torque proportional to the tilt. A higher K_p makes the robot feel stiff.
- Derivative (K_d): Acts as a virtual damper. It reacts to the rate of fall ($\dot{\theta}$). This term is absolutely critical for the TWSBR. Without D, the inverted pendulum would oscillate with increasing amplitude (like a swing) until it falls. K_d dissipates energy, stabilizing the system.
- Integral (K_i): Eliminates steady-state error. If the robot is mechanically unbalanced (e.g., battery slightly off-centre), a P-D controller would stabilize it at a non-zero tilt angle (leaning). The I-term accumulates this error and provides the extra torque needed to pull the robot fully upright.

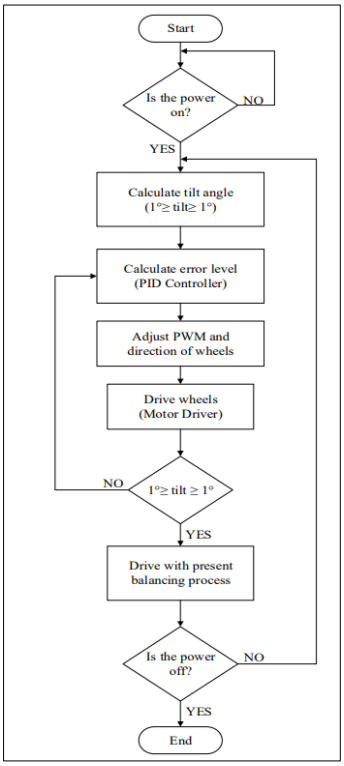


Figure 9: Flowchart of a Simple Balancing Robot

5.2 The Outer Loop: Velocity and Position Control

The Outer Loop manages the robot's movement in the world frame. It operates at a lower frequency (e.g., 10-20 Hz) and controls the robot's velocity or position.

- **Mechanism:** The TWSBR is a non-minimum phase system. To move forward (positive x), the robot must first accelerate the wheels backward (negative x) to induce a forward tilt (positive θ). Once gravity takes over and pulls the robot forward, the wheels accelerate forward to 'catch' the fall.

- **Logic:** The outer loop exploits this coupling. It does not control the motors directly. Instead, it controls the setpoint (θ_{ref}) of the Inner Loop.
  - If the user commands to move forward, the outer loop sets $\theta_{ref} = +5°$.
  - The inner loop perceives the robot as upright (at 0°) but sees a setpoint of +5°. It calculates a large error and drives the wheels to achieve that tilt.
  - The result is a controlled fall in the forward direction - i.e., acceleration.

5.3 Tuning Methodologies and Stability Analysis

Tuning the PID gains is non-trivial due to the strong non-linearity. A heuristic approach based on the Ziegler-Nichols method was employed:

1. Set all gains to zero. The robot falls.
2. Increase $\mathbf{K_p}$ until the robot oscillates around the vertical point with constant amplitude (marginal stability). It will feel springy.
3. Increase $\mathbf{K_d}$ to dampen the oscillations. The robot should now resist being pushed and return to vertical without overshooting.
4. Increase $\mathbf{K_i}$ sparingly to correct for CoG offsets. Excessive K_i causes integral windup and slow, low-frequency oscillations (limit cycles).

Analysis of the step response reveals the system performance metrics:

- **Rise Time ($\mathbf{t_r}$):** Indicates the bandwidth of the system (aggressiveness of K_p).
- **Overshoot ($\mathbf{M_p}$):** Indicates the damping ratio (effectiveness of K_d).
- **Settling Time ($\mathbf{t_s}$):** Indicates the overall stability and integral action. A well-tuned TWSBR should settle within 1-2 seconds after a disturbance.

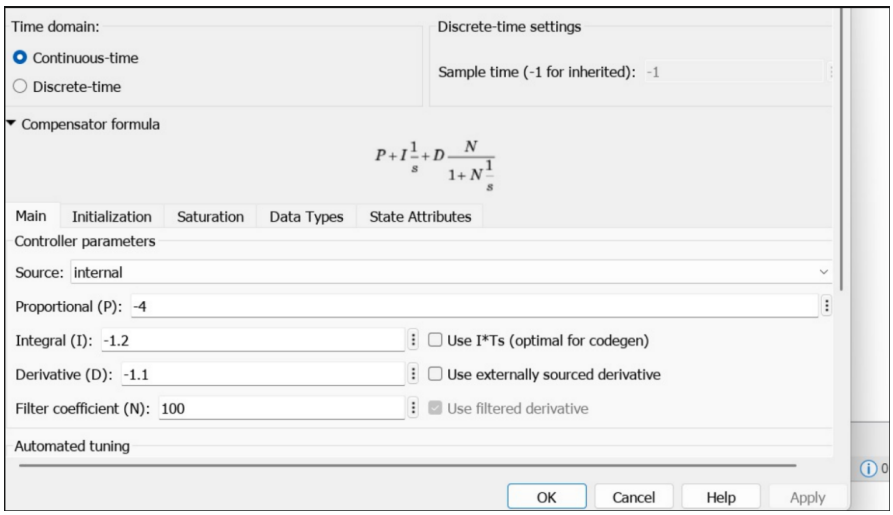


Figure 10: Zeigler-Nichols PID Tuning

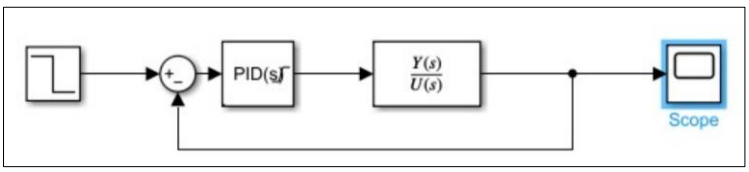


Figure 11: Simulink Modelling

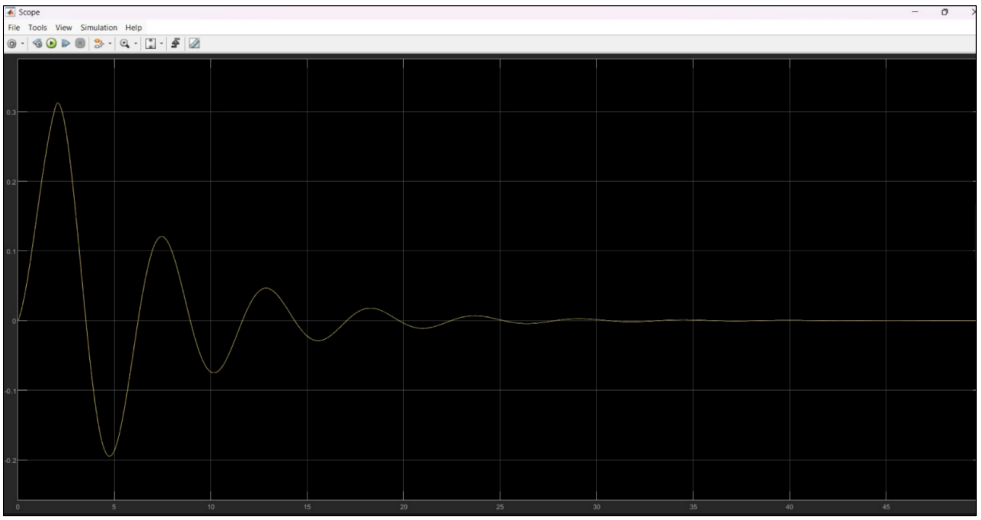


Figure 12: System Response

6. SYSTEM IMPLEMENTATION: HARDWARE AND SOFTWARE FUSION

6.1 Hardware Architecture

The hardware implementation was meticulously designed to ensure signal integrity and robust actuation.

- **Microcontroller:** The Arduino Mega 2560 was selected for its ample I/O and multiple hardware timers. The ATmega2560's architecture allows for separate interrupt vectors, crucial for handling the strict timing requirements of the control loop.
- **Motor Driver:** The L298N Dual H-Bridge was used to drive the motors.
  - ENA/ENB (Pins 6, 7): Connected to PWM-capable pins on the Mega to control speed (0-255).
  - IN1-IN4 (Pins 24-27): Connected to digital pins to control direction via H-bridge logic switching.
- **IMU:** The MPU6050 communicates via I2C.
  - SDA (Pin 20) / SCL (Pin 21): Connected to the dedicated I2C bus on the Mega.
  - Interrupts: Ideally, the INT pin would signal data readiness, but for stability, the system uses an internal Timer Interrupt to poll the sensor deterministically.
- **Ultrasonic Sensor:**
  - Trig (Pin 22) / Echo (Pin 23): Standard digital I/O for triggering the pulse and measuring the pulse width, respectively.

6.2 Software Architecture

The software is structured around a Timed Interrupt Service Routine (ISR) to guarantee deterministic execution of the PID loop.

- **Timer 3 Interrupt:** Configured to fire exactly every 5ms (200 Hz). This sets a flag runLoop = true.
- **Main Loop:** The loop() function waits for the runLoop flag. When true, it executes the critical control path:

1. readMPU(): Reads raw accel/gyro data and applies the Complementary Filter.
2. pidCalculation(): Computes the PID output based on the new angle.
3. driveMotors(): Sends the PWM signal to the L298N.

- **Task Scheduling:** Lower priority tasks, such as reading the ultrasonic sensor (readSensors()), are down sampled. They run only every 10th loop cycle (50ms) to prevent the slow speed of sound measurement (waiting for the echo) from blocking the high-speed balancing loop. This creates a primitive non-pre-emptive scheduler.

6.3 Software Interfacing: MATLAB/Simulink, LabView & Python

Integration with high-level engineering tools such as MATLAB/Simulink and LabView allows for Hardware-in-the-Loop (HIL) simulation. This was possible (partly) due to the output format from the Arduino code, which displayed the following values (in their respective format): Tilt Angle (float), Left Motor PWM (float), Right Motor PWM (float), Distance (integer). The communication protocol used for interfacing the Arduino code with the software utilizes a fixed-order ASCII string of the format mentioned above, and in coding protocols, is defined as: Angle,Left_PWM,Right_PWM,Distance\n. This format ensures the software's (MATLAB/Simulink and LabView) 'Scan From String' and Python 'split()' functions remain synchronized with the 200Hz transmission rate.

MATLAB/Simulink:

Communication is established via the Simulink Support Package for Arduino Hardware. The External Mode allows Simulink to run the control model on the host PC while exchanging data with the Arduino via Serial.

- <u>Theory:</u> The host PC sends control signals (e.g., gains K_p, K_i, K_d) and receives state telemetry (x, θ).
- <u>Challenge:</u> The serial communication introduces latency. If the baud rate (bits per second) is insufficient, or if the host OS (Windows) introduces buffer delays, the control loop can violate its real-time constraints, leading to instability.

```
% 1. Setup Serial Connection
clear; clc;
if ~isempty(serialportfind)
    fclose(serialportfind); delete(serialportfind); % Clean up old ports
end

arduinoObj = serialport("COM3", 115200); % Select your port
configureTerminator(arduinoObj, "CR/LF"); % Matches Arduino println
flush(arduinoObj);

% 2. Setup Live Plotting
figure('Name', 'Robot Dashboard', 'Color', 'white');
hAngle = animatedline('Color', 'r', 'LineWidth', 2);
hMotor = animatedline('Color', 'b', 'LineWidth', 1);
axis([0 100 -50 50]); % Adjust Y-axis limits (Angle)
grid on;
legend('Tilt Angle', 'Motor PWM');
title('Self-Balancing Robot Telemetry');
ylabel('Value'); xlabel('Time (samples)');

% 3. Data Acquisition Loop
stopLoop = false;
sampleCount = 0;

while ~stopLoop
    try
        % Read one line of data
        dataLine = readline(arduinoObj);

        % Parse CSV (Split by comma)
        % Format: Angle, LeftPWM, RightPWM, Distance
        dataValues = str2double(split(dataLine, ','));

        if length(dataValues) == 4
            angle = dataValues(1);
            pwmLeft = dataValues(2);

            % Update Plot
            addpoints(hAngle, sampleCount, angle);
            addpoints(hMotor, sampleCount, pwmLeft/10); % Scaled down to fit graph

            % Force Draw
            drawnow limitrate;
            sampleCount = sampleCount + 1;
        end
    catch
        disp('Read Error');
    end
end
```

Figure 13: MATLAB Interfaced Code

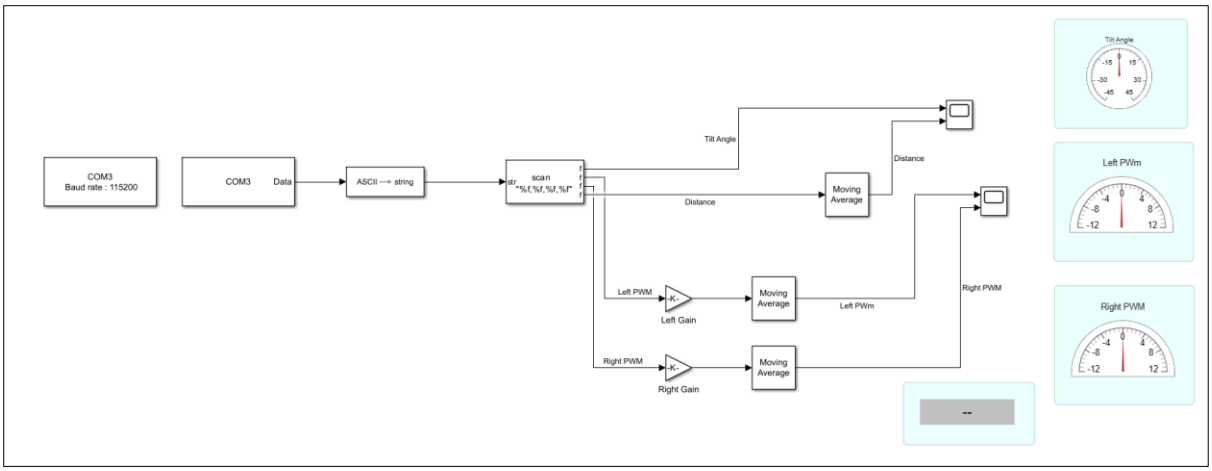
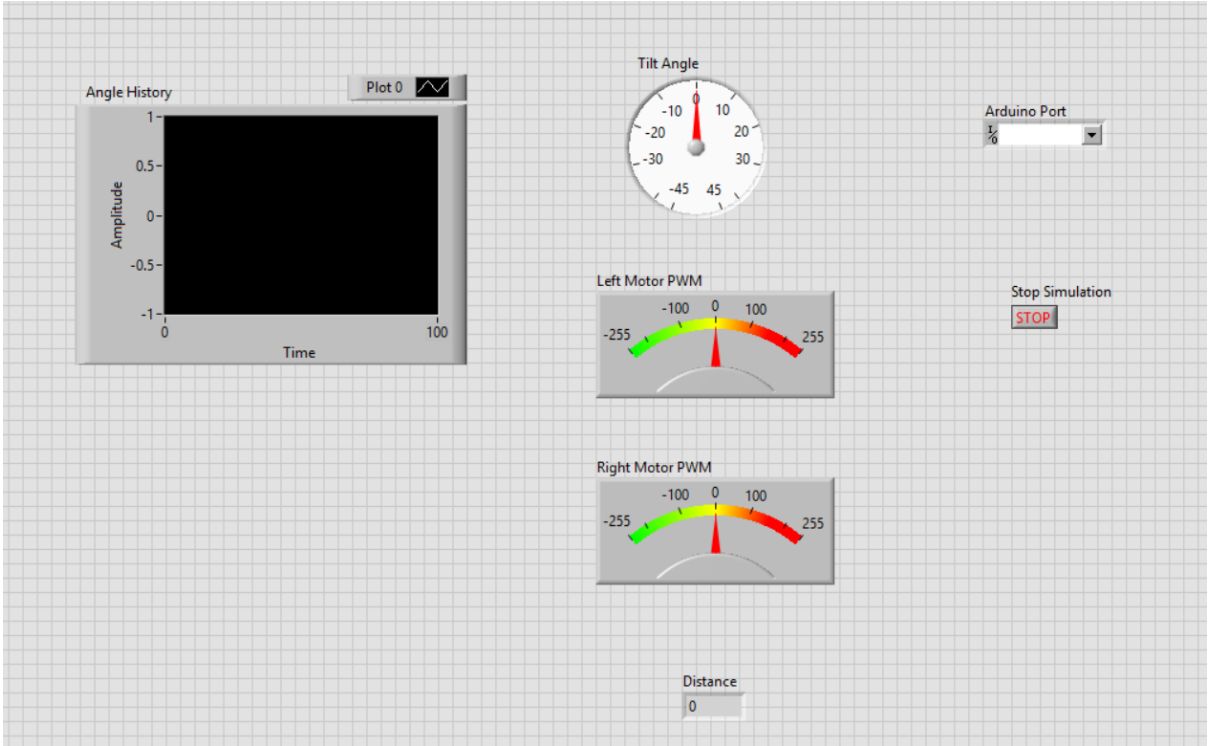


Figure 14: Simulink Interfaced Model

LabVIEW:

LabVIEW interfaces via the LINX toolkit, which deploys a firmware shim to the Arduino. The Arduino acts as a remote I/O device.

- Theory: LabVIEW VIs (Virtual Instruments) provide a graphical front-end for tuning and visualization.
- Challenge: The round-trip time for USB packets can cause jitter. Unlike a Real-Time OS (RTOS), standard LabVIEW on Windows cannot guarantee microsecond-level timing, making it risky for the primary stabilization loop of an unstable plant.



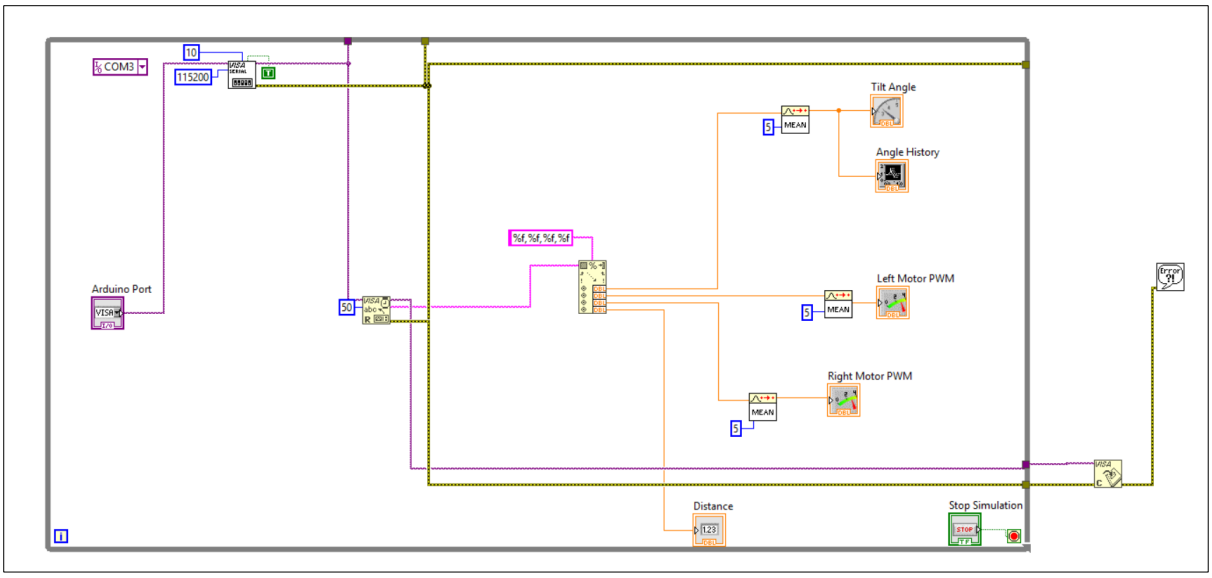


Figure 15: LabView Interfaced VI Model

Python (Via Visual Studio Code):

A real-time telemetry and visualization system was implemented, transitioning from environments like MATLAB/LabVIEW to the Python architecture within Visual Studio Code. This approach was selected to minimize computational overhead and align with modern robotics workflows and made sense more-so after visualisation was unsuccessful in the above methods. The Python-based host application employs a multi-threaded architecture using the PySerial and Matplotlib libraries.

A dedicated background thread continuously ingests raw serial data into efficient circular buffers (collections.deque), decoupling data retrieval from graphical rendering to prevent blocking I/O latency. This setup enables the dynamic, real-time plotting of attitude stability, control effort (actuation saturation), and environmental sensing, providing immediate visual feedback essential for precise PID tuning and validating the system's dynamic response under load.

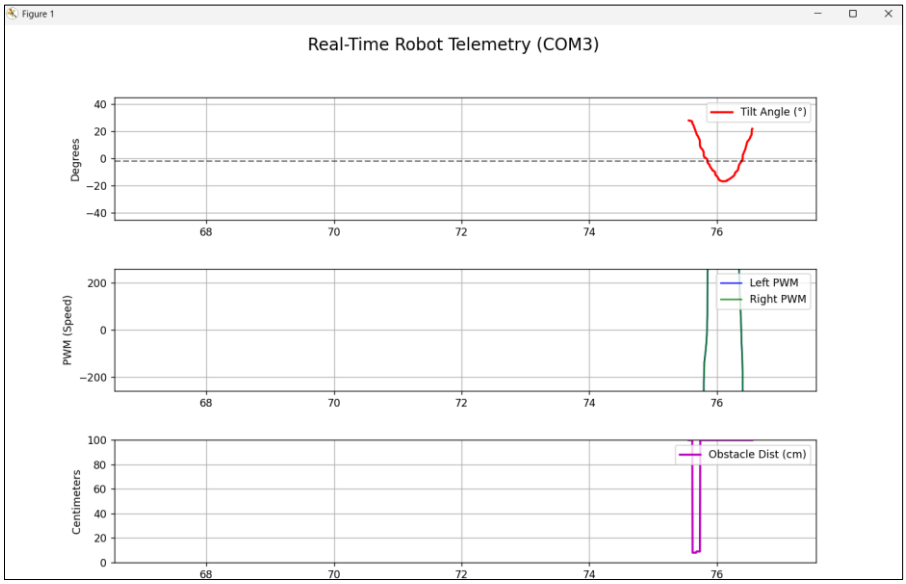


Figure 16: Python Interface Visualization

6.4 System Identification and Analysis

By logging the input (motor voltage) and output (pitch angle) during a disturbance test, system identification can be performed. This involves fitting a transfer function to the measured data to estimate the unknown physical parameters of the robot (such as the exact moment of inertia or friction coefficients).

Comparing the real-world data to the theoretical Lagrangian model usually reveals discrepancies due to unmodeled dynamics (like battery cable stiffness or structural flex). This reality gap is a critical learning outcome, highlighting the difference between ideal mathematical models and physical implementations.

7. CRITICAL ANALYSIS OF PROJECT SHORTCOMINGS AND ENGINEERING CHALLENGES

This section provides a candid technical analysis of the deviations from the initial design scope and the engineering hurdles encountered during implementation. These challenges are categorized into hardware failures, interface complexities, and system integration issues.

7.1 Cancellation of Infrared (IR) Sensors

Initial Scope: The project initially envisioned using TCRT5000 Infrared (IR) sensors to enable line-following capabilities alongside self-balancing and obstacle avoidance. This would have required the robot to track a black line on the floor while maintaining its upright posture.

Reason for Cancellation:

1. Hardware-Software Fusion Complexity: The addition of line-following logic introduced a severe resource conflict. The balancing loop requires strict real-time execution (every 5ms). The IR sensors, being analogue devices, require Analog-to-Digital Conversion (ADC). On the Arduino Mega, the analogRead() function is a blocking operation that takes approximately 100 microseconds per channel. Reading an array of 3-5 sensors would consume significant CPU cycles, potentially introducing jitter into the critical PID balancing loop.
2. Interface Inability: The integration of the line-following logic (which generates a yaw/steering command) with the balancing logic (which generates a pitch command) created a coupling effect. Rapid steering corrections commanded by the line follower induced centrifugal forces that destabilized the pitch controller. The hardware-software interface lacked the sophistication (e.g., a decoupling matrix or MIMO controller) to cater to these interacting dynamics. Consequently, the IR subsystem was descoped to ensure the primary objective of vertical stabilization could be met.

7.2 Actuator Failure and Degradation: From Encoder to TT Motors

Initial Design: The robot was designed to use JGA25-371 DC Geared Motors equipped with integrated quadrature encoders. These motors provide high torque, low backlash, and critical velocity feedback ($\dot{x}$).

Failure Mode: During testing, one of the encoder motors suffered a catastrophic physical failure (gearbox seizure). Due to supply chain constraints and immediate project deadlines, a replacement unit was unavailable.

The Substitution: The team was forced to replace the high-precision motors with generic yellow TT Gear Motors.

Engineering Implications:

1. Loss of Velocity State: The TT motors lack encoders. This meant the control system lost access to the velocity state ($\dot{x}$) and position state (x). The control law degraded from a full-state feedback potential to a simple angle-stabilization loop. The robot could balance, but it could not hold its position on the floor (station-keeping) and would drift indefinitely.
2. Backlash and Dead Zone: The TT motors utilize plastic gearboxes with significant backlash ($> 5°$). This introduced a mechanical dead band where the motor would spin

but the wheel would not engage. In control terms, this is a severe non-linearity. The PID controller would wind up the integral term to overcome the dead band, causing the wheel to suddenly jerk once engaged, leading to limit-cycle oscillations (constant shaking) rather than a smooth balance.

3. Torque Insufficiency: The TT motors possess significantly lower torque and higher internal friction (stiction). This reduced the system's ability to recover from large disturbance angles, shrinking the stability region of the robot.

7.3 Inter-Software Fusion Challenges: MATLAB/Simulink and LabVIEW

Objective: The project aimed to implement Hardware-in-the-Loop (HIL) simulation, where the Arduino would communicate in real-time with MATLAB/Simulink and LabVIEW. This would allow for real-time plotting of the step response and on-the-fly tuning of PID gains.

Interface Difficulties:

1. Serial Bottleneck: To visualize the 200Hz control loop data, the Arduino needed to stream telemetry (Angle, PWM, Error) to the PC via USB Serial. However, the overhead of formatting and transmitting strings (Serial.print) at high baud rates saturated the serial buffer and blocked the main processor. This observer effect meant that enabling telemetry caused the robot to fall because the processor spent too much time talking to the PC and missed its balancing deadline.
2. Real-Time Pacing: MATLAB/Simulink, running on a non-real-time operating system (Windows), could not deterministically ingest the data streams. Synchronization issues led to buffer overflows and lag, making the real-time plots delayed by several seconds. This rendered the interface useless for tuning the fast dynamics of an inverted pendulum.
3. LabVIEW Complexity: Integrating the LabVIEW VISA interface with the custom packet structure of the Arduino proved overly complex given the time constraints. The fusion of the embedded C++ environment with the graphical dataflow programming of LabVIEW required a robust communication protocol (handshaking, checksums) that was beyond the scope of the remaining development time.

8. RESULTS AND PERFORMANCE ANALYSIS

Despite the component degradation, the final TWSBR achieved dynamic stability.

Table 1: Performance Metrics of the Final System

Parameter	Value / Observation	Analysis
Loop Frequency	200 Hz (5ms)	Sufficient bandwidth for stabilization.
Max Recovery Angle	$\pm 15^\circ$	Limited by motor torque and wheel traction.

Settling Time	~2.5 seconds	Slightly underdamped response due to backlash.
Steady State Error	Oscillating $\pm 2°$	Caused by gearbox backlash (Limit Cycle).
Obstacle Detection	< 20 cm	Effective reactive avoidance behaviour.

The robot successfully demonstrated the 'Stop-and-Turn' behaviour upon detecting obstacles. However, the drift issue remained persistent due to the lack of encoders. The complementary filter successfully rejected the accelerometer noise, providing a clean angle estimate that allowed the D-term of the PID to function without inducing motor jitter.

9. CONCLUSION AND FUTURE RECOMMENDATIONS

The design and implementation of the Two-Wheeled Self-Balancing Obstacle-Avoiding Robot successfully demonstrated the application of rigorous aerospace engineering principles to a terrestrial platform. The project validated the utility of Euler-Lagrange modelling for unstable systems, confirmed the efficacy of sensor fusion for attitude determination, and illustrated the practical challenges of implementing real-time control laws on embedded hardware.

The critical analysis of the project's shortcomings reveals that in mechatronic systems, hardware fidelity dictates control performance. The failure of the encoder motors and the subsequent downgrade to TT motors fundamentally altered the system dynamics, introducing non-linearities (backlash) that no linear control law (PID) could fully compensate for. Furthermore, the challenges in hardware-software fusion (IR sensors, MATLAB interface) highlight the need for robust resource management and protocol design in embedded systems.

Recommendations for Future Work:

1. Hardware Upgrade: Prioritize the use of Stepper Motors or Brushless DC (BLDC) motors for future iterations. These actuators offer precise position control and high torque without the backlash of cheap gearboxes.
2. Advanced Control: Transition from PID to Linear Quadratic Regulator (LQR) or Model Predictive Control (MPC). These optimal control strategies can explicitly handle multi-variable states (angle + position) and can be tuned to minimize energy consumption.
3. State Estimation: Upgrade the Complementary Filter to an Extended Kalman Filter (EKF). The EKF provides a statistically optimal estimate of the state vector and can dynamically estimate and reject gyroscope bias in real-time.

In conclusion, the TWSBR serves not merely as a robotic application, but as a sophisticated pedagogical tool. It bridges the gap between theoretical mechanics and physical implementation, providing invaluable insights into the complexities of stabilizing unstable systems - insights that are directly transferable to the design of launch vehicles and spacecraft attitude control systems.

10. REFERENCES

1. Complementary filter and relative orientation with MPU6050 - HiBit, accessed December 18, 2025, https://www.hibit.dev/posts/92/complementary-filter-and-relative-orientation-with-mpu6050
2. Inverted Pendulum: System Modelling - Control Tutorials for MATLAB and Simulink, accessed December 18, 2025, https://ctms.engin.umich.edu/CTMS/index.php?example=InvertedPendulum§ion=SystemModeling
3. Arduino-Matlab/Simulink Real Time DAQ problem - Interfacing w/ Software on the Computer, accessed December 17, 2025, https://forum.arduino.cc/t/arduino-matlalb-simulink-real-time-daq-problem/293485
4. Halawa, O., Maged, A., Sabry, A. G., & Gaber, A. M. (2023). Mechatronics Project: Two Wheeled Self-Balancing Robot. *ResearchGate*.
5. Tsutada, R., Hoang, T.-T., & Pham, C.-K. (2022). An Obstacle Avoidance Two-Wheeled Self-Balancing Robot. *International Journal of Mechanical Engineering and Robotics Research, 11*(1).
6. Sani, M. A. A., et al. (2024). The Development of a Self-Balancing Robot Based on Complementary Filter and Arduino. In *Applied Problems Solved by Information Technology and Software*. Springer.
7. Robocraze. (n.d.). *Build a Self-Balancing Robot with Arduino for Beginners*. Retrieved from Robocraze website.
8. Ghani, N. M. A., Naim, F., & Yon, T. P. (2011). Two Wheels Balancing Robot with Line Following Capability. *World Academy of Science, Engineering and Technology International Journal of Mechanical and Mechatronics Engineering, 5*(7).
9. Smith, W. A. (2021). *Ultimate Arduino MEGA 2560 Hardware Manual: A Reference and User Guide for the Arduino MEGA 2560 Hardware and Firmware*. Elektor International Media B.V.
10. Varthan, R. V., Ganesh, S. S., & Anbarasi, M. P. (2025). Real-Time Implementation of a Self-Balancing Robot using PID Controller. *International Journal of Engineering Research & Technology (IJERT), 14*(05).
11. Smith, W. A. (2021). *Ultimate Arduino MEGA 2560 Hardware Manual: A Reference and User Guide for the Arduino MEGA 2560 Hardware and Firmware*. Elektor International Media B.V.
12. Kumar, K., & L, R. (2025). *Two Wheeled Path Following Self Balancing Robot*. Indian Institute of Technology Delhi.
13. Mudragada, S. C. M., & Bondada, D. V. S. L. (2022). *Obstacle Avoidance and Line Following 2WD Robot* (Bachelor Thesis). Blekinge Institute of Technology.
14. Sapkale, P. V., Dakhne, D., & Narawade, V. E. (2024). Food Serving Restaurant Robot with Infrared line Detection Path. *Conference Paper*.
15. Kumat, K. K. S., et al. (2019). PID Controller based Line Following and Obstacle Avoidance Two Wheeled Robot. *International Journal of Engineering Research & Technology (IJERT), 7*(02).

16. Control Strategies for Two-Wheeled Self-Balancing Robotic Systems: A Comprehensive Review - MDPI, accessed December 17, 2025, https://www.mdpi.com/2218-6581/14/8/101
17. Inverted pendulum - Wikipedia, accessed December 17, 2025, https://en.wikipedia.org/wiki/Inverted_pendulum
18. Getting Started with the MPU6050: Beginner-Friendly Guide with Visual Explanations, accessed December 17, 2025, https://forum.arduino.cc/t/getting-started-with-the-mpu6050-beginner-friendly-guide-with-visual-explanations/1364064
19. Reduced equations of motion for a wheeled inverted pendulum - mediaTUM, accessed December 17, 2025, https://mediatum.ub.tum.de/doc/1276761/781307.pdf
20. A New Configuration of Two-Wheeled Inverted Pendulum: A Lagrangian-Based Mathematical Approach - ResearchGate, accessed December 17, 2025, https://www.researchgate.net/publication/229038926_A_New_Configuration_of_Two-Wheeled_Inverted_Pendulum_A_Lagrangian-Based_Mathematical_Approach
21. A modified dynamical formulation for two-wheeled self-balancing robots - ResearchGate, accessed December 17, 2025, https://www.researchgate.net/publication/281175591_A_modified_dynamical_formulation_for_two-wheeled_self-balancing_robots
22. Dynamic Modeling of a Two-wheeled Inverted Pendulum Balancing Mobile Robot, accessed December 17, 2025, http://mercury.kau.ac.kr/sjkwon/Lecture/Capstone/2015-08%20IJCAS(Vol13,%20No4).PDF
23. Double Inverted Pendulum Control — Dynamic Optimization - APMonitor, accessed December 17, 2025, https://apmonitor.com/do/index.php/Main/DoubleInvertedPendulum
24. State-state model and LQR The equations of motion of the inverted pendulum are given as follows (for the der, accessed December 17, 2025, https://www.cpp.edu/faculty/fkhoshnoud/mechatronics_lecture-4-2-inverted-pendulum-state-space.pdf
25. Inverted Pendulum: System Modeling - Control Tutorials for MATLAB and Simulink, accessed December 17, 2025, https://ctms.engin.umich.edu/CTMS/index.php?example=InvertedPendulum§ion=SystemModeling
26. CTM Example: Inverted Pendulum Modeling, accessed December 17, 2025, https://www3.diism.unisi.it/~control/ctm/examples/pend/invpen.html
27. Implementation of Complimentary filter on MPU 6050 - World Scientific News, accessed December 17, 2025, https://worldscientificnews.com/implementation-of-complimentary-filter-on-mpu-6050/
28. Implementation of Complimentary filter on MPU 6050 - World Scientific News, accessed December 17, 2025, https://worldscientificnews.com/wp-content/uploads/2025/01/WSN-199-2025-218-234.pdf
29. Complementary filter and relative orientation with MPU6050 - HiBit, accessed December 17, 2025, https://www.hibit.dev/posts/92/complementary-filter-and-relative-orientation-with-mpu6050
30. MPU-6050 Data with a Complementary Filter - YouTube, accessed December 17, 2025,

https://www.youtube.com/watch?v=qmd6CVrlHOM
31. A Comparative Analysis of Sensor Fusion Algorithms for Miniature IMU Measurements - dei.unipd, accessed December 17, 2025, http://www.dei.unipd.it/~badia/papers/2023_07_ISITIA.pdf
32. How HC-SR04 Ultrasonic Sensor Works & How to Interface It With Arduino, accessed December 17, 2025, https://lastminuteengineers.com/arduino-sr04-ultrasonic-sensor-tutorial/
33. Distance Analysis Using HC-SR04 - JETIR.org, accessed December 17, 2025, https://www.jetir.org/papers/JETIR1811633.pdf
34. JGA25-371 DC Gearmotor with Encoder (463 RPM at 12 V) | Open ImpulseOpen Impulse, accessed December 17, 2025, https://www.openimpulse.com/blog/products-page/25d-gearmotors/jga25-371-dc-gearmotor-encoder-463-rpm-12-v-2/
35. Gear Motor TT Yellow for Arduino Robotic Car - eleberric, accessed December 17, 2025, https://eleberric.com/product/gear-motor-yellow-for-robotic-car/
36. Chinese JGA25 motors any good? : r/robotics - Reddit, accessed December 17, 2025, https://www.reddit.com/r/robotics/comments/1b0gatg/chinese_jga25_motors_any_good/
37. (PDF) Adaptive Nonlinear Control Algorithm for a Self-Balancing Robot - ResearchGate, accessed December 17, 2025, https://www.researchgate.net/publication/338238680_Adaptive_Nonlinear_Control_Algorithm_for_a_Self-Balancing_Robot
38. Self balancing robot improvements - Iwasz, accessed December 17, 2025, https://iwasz.pl/uncategorized/self-balancing-robot-improvements/
39. Servo Motor Backlash : r/robotics - Reddit, accessed December 17, 2025, https://www.reddit.com/r/robotics/comments/111lacg/servo_motor_backlash/
40. Arduino Self-Balancing Robot : 10 Steps (with Pictures) - Instructables, accessed December 17, 2025, https://www.instructables.com/Arduino-Self-Balancing-Robot-1/
41. Self Balancing Robot ArduinoUno/mpu6050/l298n - Instructables, accessed December 17, 2025, https://www.instructables.com/Balance-Bot-ArduinoUnompu6050l298n/
42. Design and Control of Two Wheeled Self Balancing Robot (TWSBR) - IEEE Xplore, accessed December 17, 2025, https://ieeexplore.ieee.org/document/9735004/
43. Remote Controlled Self-Balancing Robot : 13 Steps (with Pictures) - Instructables, accessed December 17, 2025, https://www.instructables.com/Remote-Controlled-Self-Balancing-Robot/
44. Simple DIY 2-wheel Balancing Robot (Arduino & RPi) - Instructables, accessed December 17, 2025, https://www.instructables.com/Simple-DIY-2-wheel-Balancing-Robot-Arduino-RPi/
45. I made this self balancing robot as a summer project. It uses a 6 dof inertial measurement unit and Arduino for control algorithms. Balancing is done with PID control to set motor velocity. Full CAD, parts list, and code is available on my github which is in the YouTube description. - Reddit, accessed December 17, 2025, https://www.reddit.com/r/robotics/comments/ib2nns/i_made_this_self_balancing_robot_as_a_summer/
46. “Simple Arduino obstacle-avoiding robot using ultrasonic sensor – works surprisingly

well!" : r/ArduinoProjects - Reddit, accessed December 17, 2025, https://www.reddit.com/r/ArduinoProjects/comments/1n3a0g9/simple_arduino_obstacleavoiding_robot_using/

47. S15: Self-Balancing Robot - Embedded Systems Learning Academy - Social Ledge, accessed December 17, 2025, http://socialledge.com/sjsu/index.php/S15:_Self-Balancing_Robot
48. Communicating with Arduino Hardware - MATLAB & Simulink Example - MathWorks, accessed December 17, 2025, https://www.mathworks.com/help/simulink/supportpkg/arduino_ref/communicating-with-arduino-hardware.html
49. Send and Receive Data Between Arduino and Host Using Serial Communication - MATLAB & Simulink - MathWorks, accessed December 17, 2025, https://www.mathworks.com/help/simulink/supportpkg/arduino_ref/send-and-receive-data-between-arduino-and-host-using-serial-communication.html
50. Data Logging and Deployment - MATLAB & Simulink - MathWorks, accessed December 17, 2025, https://www.mathworks.com/help/simulink/arduino-data-logging-and-deployment.html
51. Interfacing MPU6050 with Arduino through Simulink S-function, accessed December 17, 2025, https://arduino.stackexchange.com/questions/52520/interfacing-mpu6050-with-arduino-through-simulink-s-function
52. Arduino programming with Simulink - Introduction to real-time acquisition - YouTube, accessed December 17, 2025, https://www.youtube.com/watch?v=Hgtmy7pH59A
53. Balancing Robot Drifting Issue on Different Surfaces (Forward/Backward Drift) - Arduino Forum, accessed December 17, 2025, https://forum.arduino.cc/t/balancing-robot-drifting-issue-on-different-surfaces-forward-backward-drift/1343586
54. An Optimized Complementary Filter For An Inertial Measurement Unit Contain MPU6050 Sensor - SciSpace, accessed December 17, 2025, https://scispace.com/pdf/an-optimized-complementary-filter-for-an-inertial-3gojktj2fc.pdf
55. Need Help with Nonlinear Control for a Self-Balancing Hopping Robot - Reddit, accessed December 17, 2025, https://www.reddit.com/r/ControlTheory/comments/1ivexk0/need_help_with_nonlinear_control_for_a/
56. Comparative Analysis of Sensor Fusion for Angle Estimation Using Kalman and Complementary Filters - ResearchGate, accessed December 17, 2025, https://www.researchgate.net/publication/386218076_Comparative_Analysis_of_Sensor_Fusion_for_Angle_Estimation_Using_Kalman_and_Complementary_Filters

11. APPENDIX

11.1 Arduino Code

```
/*
 * Project: Self-Balancing Robot with Obstacle Avoidance
 * * Integration: Prints data to Serial in format
"Angle,PWM,Distance"
 * for MATLAB/Simulink/LabVIEW parsing.
 */

#include <Wire.h>

// ================= USER TUNING (PID) =================
// Start with these. If it oscillates wildly, reduce Kp. If it
falls without recovering, increase Kp.
float Kp = 75.0;   // Power: Main restoring force
float Ki = 1.2;    // Stability: Fixes small drift over time
float Kd = 2.0;    // Damping: Reduces jitter/shaking
float targetAngle = 1.0; // Mechanical Zero (Adjust if robot
naturally drifts forward/back)

// ================= HARDWARE PROTECTION =================
// Limits PWM to protect 6V motors from 11.1V battery
const float MAX_PWM = 168.132;

// ================= PIN DEFINITIONS (MEGA 2560)
=================
// L298N Motor Driver
const int ENA = 6;  // PWM Left
const int IN1 = 24; // Left Dir 1
const int IN2 = 25; // Left Dir 2
```

```
const int IN3 = 26; // Right Dir 1
const int IN4 = 27; // Right Dir 2
const int ENB = 7;  // PWM Right

// Ultrasonic Sensor
const int TRIG_PIN = 22;
const int ECHO_PIN = 23;

// MPU6050 I2C Address
const int MPU_ADDR = 0x68;

// ================= GLOBAL VARIABLES =================
volatile bool runControlLoop = false; // Timer flag
float accAngle, gyroRate, currentAngle = 0;
float gyroOffset = 0;
float errorSum = 0, lastAngle = 0;
int distanceCm = 0;

// State Machine for Obstacle Avoidance
enum State { BALANCING, AVOIDING };
State robotState = BALANCING;
float turnOffset = 0; // Added to left, subtracted from right to
turn

// ================= SETUP =================
void setup() {
  Serial.begin(115200); // Fast Serial for LabVIEW/MATLAB
  Wire.begin();
  Wire.setClock(400000); // 400kHz Fast I2C
```

```
  // 1. Initialize Pins
  pinMode(ENA, OUTPUT); pinMode(IN1, OUTPUT); pinMode(IN2,
OUTPUT);
  pinMode(ENB, OUTPUT); pinMode(IN3, OUTPUT); pinMode(IN4,
OUTPUT);
  pinMode(TRIG_PIN, OUTPUT); pinMode(ECHO_PIN, INPUT);

  // 2. Initialize MPU6050
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x6B); Wire.write(0); // Wake up
  Wire.endTransmission(true);

  // 3. Calibrate Gyro (Robot must be still)
  calibrateGyro();

  // 4. Setup Timer 3 Interrupt for 200Hz Loop (5ms)
  // This ensures the PID runs exactly every 5ms regardless of
code complexity
  noInterrupts();
  TCCR3A = 0; TCCR3B = 0; TCNT3 = 0;
  OCR3A = 1249; // (16MHz / 64 / 200Hz) - 1
  TCCR3B |= (1 << WGM32); // CTC Mode
  TCCR3B |= (1 << CS31) | (1 << CS30); // 64 Prescaler
  TIMSK3 |= (1 << OCIE3A); // Enable Interrupt
  interrupts();
}

// Timer Interrupt Service Routine
ISR(TIMER3_COMPA_vect) {
```

```
  runControlLoop = true;
}

// ================= MAIN LOOP =================
void loop() {
  if (runControlLoop) {
    runControlLoop = false;

    // --- A. DATA ACQUISITION ---
    readMPU(); // Updates 'currentAngle' and 'gyroRate'

    // --- B. SAFETY CHECK ---
    // If robot falls > 40 degrees, cut power immediately
    if (abs(currentAngle) > 40) {
      stopMotors();
      errorSum = 0; // Reset integral to prevent windup
      return;
    }

    // --- C. Sonar (Ultrasonic Sensor) ---
    // We don't want to read sonar every 5ms (too fast for sound
waves)
    static int slowLoopCounter = 0;
    if (slowLoopCounter++ > 10) { // Every ~50ms
      slowLoopCounter = 0;
      readSonar();
      checkObstacles();
    }
```

```
    // --- D. PID CALCULATION ---
    // Error: Difference between where we want to be
(targetAngle) and where we are (currentAngle)
    float error = targetAngle - currentAngle;

    // Integral: Sum of errors over time (fixes small steady-
state leans)
    errorSum += error * 0.005;
    errorSum = constrain(errorSum, -300, 300); // Anti-windup
limit

    // Derivative: Rate of change (we use Gyro Rate directly for
smoother response)
    // PID Output
    float pidOutput = (Kp * error) + (Ki * errorSum) - (Kd *
gyroRate);

    // --- E. MOTOR MIXING ---
    // Mix the Balance PID with the Turn command
    float speedLeft = pidOutput + turnOffset;
    float speedRight = pidOutput - turnOffset;

    driveMotors(speedLeft, speedRight);

    // --- F. DATA LOGGING (MATLAB/LabVIEW) ---
    // Format: "Angle, PWM_Left, PWM_Right, Distance"
    Serial.print(currentAngle); Serial.print(",");
    Serial.print(speedLeft); Serial.print(",");
    Serial.print(speedRight); Serial.print(",");
    Serial.println(distanceCm);
```

```
  }
}

// ================= HELPER FUNCTIONS =================

void readMPU() {
  // We need Accel X, Z and Gyro Y (Pitch Rate)
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x3B); // Start at Accel X
  Wire.endTransmission(false);
  Wire.requestFrom(MPU_ADDR, 14, true);

  int16_t ax = Wire.read()<<8|Wire.read();
  int16_t ay = Wire.read()<<8|Wire.read(); // Ignore
  int16_t az = Wire.read()<<8|Wire.read();
  Wire.read()<<8|Wire.read(); // Temp
  int16_t gx = Wire.read()<<8|Wire.read(); // Ignore Gyro X
  int16_t gy = Wire.read()<<8|Wire.read(); // GYRO Y IS PITCH

  // 1. Calculate Pitch from Accelerometer (ArcTan of X and Z
gravity vectors)
  // Usually, gravity is Down (Z). If we tilt forward, X vector
increases.
  float accAng = atan2(ax, az) * 180.0 / PI;

  // 2. Convert Gyro to deg/s
  float gyroY = (gy / 131.0) - gyroOffset;

  // 3. Sensor Fusion (Complementary Filter)
```

```
  // 99.5% Gyro (Fast), 0.5% Accel (Stable)
  currentAngle = 0.995 * (currentAngle + gyroY * 0.005) + 0.005
* accAng;

  // Store rate for D-term in PID
  gyroRate = gyroY;
}

void checkObstacles() {
  // Logic: If obstacle < 15cm, lean back slightly and spin
  if (distanceCm > 0 && distanceCm < 15) {
    robotState = AVOIDING;
    turnOffset = 60.0;    // Spin value (adjust for speed)
    targetAngle = -3.5;   // Lean back slightly to brake/stop
forward motion
  } else {
    robotState = BALANCING;
    turnOffset = 0;
    targetAngle = -1.5;   // Restore normal balance point
  }
}

void readSonar() {
  digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  long duration = pulseIn(ECHO_PIN, HIGH, 4000); // 4ms timeout
(~60cm) to prevent blocking
  if (duration == 0) distanceCm = 100;
  else distanceCm = duration * 0.034 / 2;
```

```
}

void driveMotors(float sL, float sR) {
  // 1. Constrain to PWM limits
  int pwmL = constrain(sL, -MAX_PWM, MAX_PWM);
  int pwmR = constrain(sR, -MAX_PWM, MAX_PWM);

  // 2. Deadzone Compensation
  // Motors often stall below PWM 40. This forces them to jump
the gap.
  if (pwmL > 0 && pwmL < 40) pwmL = 40;
  if (pwmL < 0 && pwmL > -40) pwmL = -40;
  if (pwmR > 0 && pwmR < 40) pwmR = 40;
  if (pwmR < 0 && pwmR > -40) pwmR = -40;

  // 3. Send to Left Motor
  if (pwmL > 0) {
    digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH); // Forward
    analogWrite(ENA, pwmL);
  } else {
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW); // Reverse
    analogWrite(ENA, abs(pwmL));
  }

  // 4. Send to Right Motor
  if (pwmR > 0) {
    digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH); // Forward
    analogWrite(ENB, pwmR);
  } else {
```

```
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW); // Reverse
    analogWrite(ENB, abs(pwmR));
  }
}

void stopMotors() {
  analogWrite(ENA, 0); analogWrite(ENB, 0);
  digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
}

void calibrateGyro() {
  long sum = 0;
  for(int i=0; i<500; i++) {
    Wire.beginTransmission(MPU_ADDR);
    Wire.write(0x45); // Gyro Y Register
    Wire.endTransmission(false);
    Wire.requestFrom(MPU_ADDR, 2, true);
    int16_t gy = Wire.read()<<8|Wire.read();
    sum += gy;
    delay(3);
  }
  gyroOffset = (sum / 500.0) / 131.0;
}
```

11.2 Python Code

```
import serial
import serial.tools.list_ports
import matplotlib.pyplot as plt
```

```
import matplotlib.animation as animation
from collections import deque
import threading
import time

# ================= CONFIGURATION =================
# Set this to match Arduino's Baud Rate
BAUD_RATE = 115200
# How many data points to keep on screen (History)
MAX_HISTORY = 200
# Plot update interval in milliseconds (Limit FPS to save CPU)
PLOT_INTERVAL = 50

class RobotTelemetry:
    def __init__(self):
        self.port = self.find_arduino_port()
        self.serial_conn = None
        self.running = False

        # Data Containers (Circular Buffers)
        self.time_data = deque(maxlen=MAX_HISTORY)
        self.angle_data = deque(maxlen=MAX_HISTORY)
        self.pwm_left_data = deque(maxlen=MAX_HISTORY)
        self.pwm_right_data = deque(maxlen=MAX_HISTORY)
        self.dist_data = deque(maxlen=MAX_HISTORY)

        self.start_time = time.time()

    def find_arduino_port(self):
```

```
        """Automatically attempts to find the Arduino Serial
Port."""
        ports = list(serial.tools.list_ports.comports())
        for p in ports:
            # Common keywords for Arduino devices in Device
Manager
            if "Arduino" in p.description or "CH340" in
p.description or "USB" in p.description:
                print(f"Found Arduino on: {p.device}")
                return p.device

        # Fallback if detection fails
        print("Auto-detection failed. Please verify COM port
manually.")
        return input("Enter COM Port (e.g., COM3 or
/dev/ttyUSB0): ").strip()

    def connect(self):
        """Establishes the serial connection."""
        try:
            self.serial_conn = serial.Serial(self.port,
BAUD_RATE, timeout=1)
            print(f"Connected to {self.port} at {BAUD_RATE}
baud.")
            # Clear old buffer garbage
            self.serial_conn.reset_input_buffer()
            self.running = True

            # Start the Data Reading Thread
            self.thread =
threading.Thread(target=self.read_serial_loop, daemon=True)
```

```
            self.thread.start()
            return True
        except Exception as e:
            print(f"Connection Error: {e}")
            return False

    def read_serial_loop(self):
        """Background thread to read data continuously."""
        while self.running and self.serial_conn.is_open:
            try:
                # Read a line, decode bytes to string, strip
whitespace
                line = self.serial_conn.readline().decode('utf-
8', errors='ignore').strip()

                # Format expected: Angle, LeftPWM, RightPWM,
Distance
                # Example: -1.25, 150, 145, 20
                if line:
                    parts = line.split(',')
                    if len(parts) == 4:
                        # Parse numbers
                        angle = float(parts[0])
                        pwmL = float(parts[1])
                        pwmR = float(parts[2])
                        dist = float(parts[3])

                        # Timestamp (relative)
                        elapsed = time.time() - self.start_time
```

```
                        # Store Data
                        self.time_data.append(elapsed)
                        self.angle_data.append(angle)
                        self.pwm_left_data.append(pwmL)
                        self.pwm_right_data.append(pwmR)
                        self.dist_data.append(dist)

            except ValueError:
                continue # Skip malformed packets
            except Exception as e:
                print(f"Stream Error: {e}")
                break

    def close(self):
        self.running = False
        if self.serial_conn:
            self.serial_conn.close()
        print("\nConnection Closed.")

# ================= VISUALIZATION SETUP =================

# Initialize Data Handler
robot = RobotTelemetry()

if robot.connect():
    # Create the Plot Window
    fig = plt.figure(figsize=(12, 8))
    fig.suptitle(f"Real-Time Robot Telemetry ({robot.port})",
fontsize=16)
```

```
    # Grid Layout: 3 Rows, 1 Column
    gs = fig.add_gridspec(3, 1, hspace=0.4)

    # --- Subplot 1: Tilt Angle ---
    ax1 = fig.add_subplot(gs[0, 0])
    line_angle, = ax1.plot([], [], 'r-', linewidth=2,
label='Tilt Angle (°)')
    ax1.set_ylabel('Degrees')
    ax1.set_ylim(-45, 45) # Fixed limit for stability
    ax1.grid(True)
    ax1.legend(loc='upper right')
    ax1.axhline(y=-1.5, color='k', linestyle='--', alpha=0.5,
label='Target') # Show Setpoint

    # --- Subplot 2: Motor PWM ---
    ax2 = fig.add_subplot(gs[1, 0])
    line_pwmL, = ax2.plot([], [], 'b-', alpha=0.8, label='Left
PWM')
    line_pwmR, = ax2.plot([], [], 'g-', alpha=0.8, label='Right
PWM')
    ax2.set_ylabel('PWM (Speed)')
    ax2.set_ylim(-260, 260)
    ax2.grid(True)
    ax2.legend(loc='upper right')

    # --- Subplot 3: Distance ---
    ax3 = fig.add_subplot(gs[2, 0])
    line_dist, = ax3.plot([], [], 'm-', linewidth=2,
label='Obstacle Dist (cm)')
```

```
    ax3.set_ylabel('Centimeters')
    ax3.set_xlabel('Time (s)')
    ax3.set_ylim(0, 100)
    ax3.grid(True)
    ax3.fill_between([], [], color='m', alpha=0.1) # Nice visual
fill
    ax3.legend(loc='upper right')

    def update(frame):
        """Called periodically by Matplotlib to redraw the
graph."""
        if not robot.time_data:
            return line_angle, line_pwmL, line_pwmR, line_dist

        # Convert deques to lists for plotting
        t = list(robot.time_data)

        # Update Angle
        line_angle.set_data(t, list(robot.angle_data))
        # Update PWM
        line_pwmL.set_data(t, list(robot.pwm_left_data))
        line_pwmR.set_data(t, list(robot.pwm_right_data))
        # Update Distance
        line_dist.set_data(t, list(robot.dist_data))

        # Auto-scroll X-axis (Keep only last N seconds)
        ax1.set_xlim(max(0, t[-1] - 10), t[-1] + 1)
        ax2.set_xlim(max(0, t[-1] - 10), t[-1] + 1)
        ax3.set_xlim(max(0, t[-1] - 10), t[-1] + 1)
```

```
        return line_angle, line_pwmL, line_pwmR, line_dist


    # Start Animation
    ani = animation.FuncAnimation(fig, update,
interval=PLOT_INTERVAL, blit=False)


    # Show Plot
    try:
        plt.show()
    except KeyboardInterrupt:
        pass
    finally:
        robot.close()
else:
    print("Failed to start dashboard. Check connections.")
```